



AI-POWERED DEVELOPER PRODUCTIVITY FOR SOFTWARE DEVELOPERS

A practical guide to using ChatGPT, GitHub Copilot, and
Codex throughout the software development lifecycle

43 CHAPTERS / 8 PARTS / CURRICULUM VERSION 1.0.0

EDITION & RESPONSIBLE USE

Copyright & Disclaimer

AI-Powered Developer Productivity for Software Developers

Copyright © 2026 **Chukwuemeka Brian Nkama**. All rights reserved.

This book is an educational resource for software developers and is provided for informational and learning purposes. It is not a substitute for professional engineering judgment, security review, legal advice, or other specialized professional advice.

AI-Generated Content and Verification

This book discusses the use of artificial intelligence in software development, including tools such as **ChatGPT**, **GitHub Copilot**, and **OpenAI Codex**.

AI-generated output can be inaccurate, incomplete, insecure, outdated, or unsuitable for a particular project. Developers should review AI-generated code and technical claims carefully, test implementations appropriately, consult current authoritative documentation, and apply their own engineering judgment before using AI-generated output in real projects.

Examples, prompts, workflows, and code presented in this book are provided for educational purposes. They should be adapted to the specific requirements, architecture, dependencies, security standards, compliance obligations, and development practices of the project in which they are used.

Software libraries, APIs, AI models, platforms, interfaces, pricing, capabilities, and recommended practices can change over time. Always consult the relevant official documentation for the current behavior and requirements of the tools you use.

The author does not guarantee that every example, recommendation, prompt, or workflow will produce a particular result in every environment.

Privacy and Confidential Information

Do not provide passwords, API keys, authentication tokens, private customer information, proprietary source code, confidential business information, personal data, or other restricted material to an AI service unless you are authorized to do so and the relevant service and organizational policies permit that use.

Before using AI tools with project data, understand the applicable privacy, security, retention, and data-use policies of both your organization and the AI service.

Reader Responsibility

Readers are responsible for evaluating and validating any code, architecture, configuration, workflow, or technical recommendation before applying it to their own projects.

The author and BrianE-Dev are not responsible for losses, security incidents, service interruptions, data loss, or other consequences resulting from the application of information contained in this book.

Course and Curriculum Information

This book is the written companion to the **BrianE-Dev developer productivity course**.

The course title and chapter structure in this edition correspond to the **BrianE-Dev curriculum, version 1.0.0**. Future editions of the course or book may contain updated content, examples, tools, or curriculum structures.

Trademarks and Third-Party Products

ChatGPT, GitHub Copilot, OpenAI Codex, and other product, company, platform, and service names mentioned in this book are the property of their respective owners.

Mention of a third-party product or service does not imply endorsement by, or affiliation with, the author or BrianE-Dev unless explicitly stated.

BRIANE-DEV

About the Author

Chukwuemeka Brian Nkama is a Full-Stack Engineer, software developer, and technology builder focused on creating practical digital products and helping developers work more effectively with modern software tools and artificial intelligence.

Through **BrianE-Dev**, he creates developer-focused learning resources designed to help software developers build stronger workflows, improve their productivity, and make better use of AI throughout the software development lifecycle.

His experience spans frontend and backend development, including **React, JavaScript, TypeScript, Node.js, Express, PostgreSQL, MongoDB, and modern web development tools and platforms**. He has worked on web applications, developer-focused platforms, business systems, and digital products, gaining practical experience across planning, development, debugging, deployment, and product delivery.

Chukwuemeka is particularly interested in the intersection of **software engineering and artificial intelligence**—not simply using AI to generate code, but understanding how developers can use AI thoughtfully across the entire development process: from planning and architecture to implementation, debugging, testing, documentation, and professional delivery.

He is also the founder and engineering lead behind **WEBILOXEN**, a technology venture studio focused on building and launching digital products.

This book represents his practical approach to AI-assisted software development and serves as the written companion to the **BrianE-Dev developer productivity course**. Its goal is to help developers move beyond simply knowing that AI tools exist and learn how to integrate them effectively into real software engineering workflows.

THE BOOK

About the Book

AI-Powered Developer Productivity for Software Developers is a practical guide to using artificial intelligence as a productivity tool throughout the software development lifecycle.

AI has changed the way software developers can approach their work. Tools such as **ChatGPT, GitHub Copilot, and OpenAI Codex** can help developers research, plan, write, understand, debug, test, document, and improve software. However, getting useful results from these tools requires more than knowing how to ask an AI to write code.

This book focuses on the **workflow, reasoning, and practices behind effective AI-assisted software engineering**. It explores how developers can integrate AI into their existing development process while maintaining technical judgment, code quality, security, maintainability, and ownership of the final product.

Throughout the book, you will learn how to use AI across different stages of software development—from understanding requirements and planning projects to implementation, debugging, testing, documentation, code review, and professional delivery.

The book covers practical workflows for working with **ChatGPT, GitHub Copilot, and Codex**, while also addressing the broader principles that apply to AI-assisted development regardless of the specific tool being used.

Rather than presenting AI as a replacement for software engineering knowledge, this book treats it as a **force multiplier for developers**. The goal is to help you become more deliberate, efficient, and capable in the way you use AI while keeping engineering decisions in your hands.

This book is designed for developers who want to move beyond experimenting with AI and build a **repeatable, professional AI-powered developer workflow** that can support real software projects.

GETTING STARTED

Who This Book Is For

This book is specifically written for:

- **Junior developers** who want to become more productive and learn how experienced developers approach problems with AI assistance.
- **Mid-level developers** who want to integrate AI into their existing development workflow without becoming overly dependent on it.
- **Frontend developers** working with technologies such as React, JavaScript, and TypeScript who want to accelerate implementation, debugging, testing, and UI development.
- **Backend developers** building APIs, services, databases, and server-side applications who want to use AI for development, debugging, documentation, and problem-solving.
- **Full-stack developers** who need to move efficiently between frontend, backend, databases, APIs, and deployment.
- **Developers transitioning into AI-assisted development** who understand traditional software development but want a structured way to adopt modern AI tools.
- **Self-taught developers and coding bootcamp graduates** who can build applications but want to develop a more professional and systematic engineering workflow.
- **Developers building personal projects, freelance projects, startups, or production applications** who want to use AI to increase their development velocity while maintaining quality.

Prerequisites

- Basic familiarity with a programming language and the purpose of source code.
- Some experience reading or changing a small software project is helpful.
- Access to an AI tool is useful for practice but is not required to read the book.

No AI tool is treated as an authority.

Use the material to improve how you frame tasks, gather context, inspect suggestions, and verify outcomes.

LEARNING PLAN

What You Will Gain

By the end of the book, you should be able to approach AI as more than a code-generation tool. You will have a practical framework for using AI as a **development assistant, research partner, debugging aid, reviewer, and productivity multiplier**.

The objective is not to let AI do your job for you. The objective is to help you **do your job better**.

This book is about developing the skills, workflows, and judgment required to make AI a valuable part of modern software engineering.

Learning Objectives

- Choose an appropriate AI tool for a bounded software-development task.
- Provide useful context and constraints while protecting private information.
- Apply AI across planning, coding, debugging, review, testing, and maintenance.
- Verify claims and code using project evidence, tests, and engineering judgment.

How to Use This Book

1. Read the chapter objectives and identify the task they describe.
2. Study the workflow and example, adapting prompts to a safe, permitted project context.
3. Complete the practice activity and make the verification evidence explicit.
4. Answer the knowledge-check questions in the paid course reader, where responses are graded server-side.
5. Return to the relevant chapter when a similar engineering task arises.

The parts build from foundational collaboration into tool-specific work, project planning, delivery, professional practice, and a repeatable personal workflow.

LEARNING PLAN

Learning Objectives

- Choose an appropriate AI tool for a bounded software-development task.
- Provide useful context and constraints while protecting private information.
- Apply AI across planning, coding, debugging, review, testing, and maintenance.
- Verify claims and code using project evidence, tests, and engineering judgment.

How to Use This Book

1. Read the chapter objectives and identify the task they describe.
2. Study the workflow and example, adapting prompts to a safe, permitted project context.
3. Complete the practice activity and make the verification evidence explicit.
4. Answer the knowledge-check questions in the paid course reader, where responses are graded server-side.
5. Return to the relevant chapter when a similar engineering task arises.

The parts build from foundational collaboration into tool-specific work, project planning, delivery, professional practice, and a repeatable personal workflow.

COURSE & CERTIFICATE

Course Information

The BrianE-Dev course contains eight sections and 43 canonical chapters. This ebook is included with paid course access and is generated from the same lesson content used by the course reader.

Course completion is recorded by the learning platform against stable chapter IDs. A Certificate of Completion is issued when the authenticated learner has a verified course purchase, completes all 43 chapters, and satisfies every required assessment and exercise.

Certificate verification

Issued certificates include a unique ID and can be checked through the public BrianE-Dev certificate verification endpoint. A certificate is not a statement of accreditation or a degree.

To follow chapter navigation, complete assessments, record progress, access the ebook, or download an issued certificate, sign in to the BrianE-Dev learner space.

Contents

Part 1 — Modern AI Developer Workflow

The AI-Assisted Developer	01
Choosing the Right AI Tool	02
Prompting for Software Development	03
When NOT to Use AI	04

Part 2 — ChatGPT for Developers

ChatGPT as a Development Assistant	05
Debugging with ChatGPT	06
Refactoring and Improving Existing Code	07
Frameworks, Libraries, and Documentation	08
Understanding Errors and Stack Traces	09
APIs, SQL, and Development Questions	10
Architecture and Technical Reasoning	11

Part 3 — GitHub Copilot

Getting Started with GitHub Copilot	12
Intelligent Autocomplete	13
Generating Code with Copilot	14
Writing Tests with Copilot	15
Refactoring with Copilot	16
Copilot as a Pair Programmer	17

Part 4 — Codex for Software Engineering

Understanding Codex	18
Generating and Editing Code	19
Multi-File Changes	20
Explaining Existing Codebases	21
Handling Larger Development Tasks	22
ChatGPT vs GitHub Copilot vs Codex	23

Contents continued

Part 5 — Planning the Project with AI

Planning a Software Project with ChatGPT	24
Turning Requirements into Tasks	25
Designing Features and User Flows	26
Choosing Technologies and Architecture	27
Creating a Development Roadmap	28
Using AI During Implementation	29

Part 6 — Building, Debugging, and Testing with AI

Generating Application Code	30
Working with Frontend Development	31
Working with Backend Development	32
Working with Databases	33
API Integration and Testing	34
Debugging Complex Problems	35
Automated Testing and Quality Assurance	36

Part 7 — Professional AI-Assisted Software Engineering

Code Review with AI	37
Documentation with AI	38
Security, Privacy, and Responsible AI Use	39
Git, Version Control, and AI-Assisted Workflows	40
Maintaining and Improving AI-Assisted Codebases	41

Part 8 — Building Your AI-Powered Developer Workflow

Designing Your Personal AI Development Workflow	42
The AI-Powered Developer: Putting Everything Together	43

The AI-Assisted Developer

Learning objectives

- Explain how AI can support software development without replacing engineering judgment.
- Choose a bounded development task that is appropriate for AI assistance.
- Review AI-generated suggestions against project context and evidence.

AI as a collaborator in engineering work

An AI-assisted developer brings a model into a workflow to speed up exploration, drafting, and analysis. The developer still owns the goal, supplies relevant context, checks the result, and decides whether it belongs in the codebase.

Treat a model response as a proposal that needs review, not as evidence that the work is correct.

Engineering review principle

A repeatable working loop

1. State the task and the outcome you need.
2. Give the model the smallest useful project context, including constraints.
3. Ask for a proposal or a small change that can be reviewed.
4. Check the reasoning, inspect the diff, and run relevant verification.
5. Keep, revise, or reject the suggestion based on evidence.

Keep the task bounded

A narrow request such as explaining one function or drafting a focused test is easier to evaluate than asking a model to redesign an unfamiliar application all at once.

Give useful context and constraints

A useful request names the relevant files or behavior, describes the expected outcome, and states constraints such as supported runtime versions or existing patterns. Avoid sharing secrets, private user data, or proprietary material unless the tool and organization explicitly permit it.

- Include the error message or failing behavior when asking for debugging help.
- Provide only the code and documentation needed for the current task.

- Ask the model to identify assumptions and suggest checks when uncertainty remains.

task-brief.txt

A task brief that asks for an inspectable first step.

Goal: Explain why `parseUser` rejects a valid-looking email.
 Context: `src/users/parseUser.js` and the failing input below.
 Constraints: Do not change the public return shape.
 Request: Trace the validation path, identify the likely cause, and suggest a focused test before proposing a fix.

Task	Useful model contribution	Developer check
Understand unfamiliar code	Summarize control flow and dependencies	Compare the explanation with the actual call sites
Draft a test	Suggest cases and an initial test structure	Check assertions against the intended behavior
Debug a failure	Offer hypotheses from logs and code context	Reproduce the failure and verify the root cause

Review before accepting changes

Generated code can be incomplete, insecure, incompatible with project conventions, or based on an incorrect assumption. Read every changed line and run checks that match the risk of the change.

Practice: review an AI suggestion

Choose a small task from a project you are allowed to share with your selected AI tool. Ask for a proposal, then list what you would inspect or test before using it.

Practice

Plan a reviewable AI-assisted task Optional

Practice defining a bounded task and a concrete review step.

1. Choose a routine development task that does not require sharing restricted information.
2. Write a short task brief with the goal, relevant context, and constraints.
3. Name one way you will independently verify the result.

Constraints: Do not include credentials, private user data, or code you are not permitted to share.

Expected outcome: A task brief and a verification step that make the model's contribution reviewable.

Knowledge check

Select your answers before checking them in the course reader. The answer key is intentionally not included in this ebook.

Question 1 (required). After an AI tool proposes a code change, what should the developer do before accepting it?

- A. Accept it because the model produced a complete response.
- B. Inspect the change and verify it against the task and project behavior.
- C. Provide credentials so the model can make a more confident change.

Choosing the Right AI Tool

Learning objectives

- Explain how Choosing the Right AI Tool supports a software-development task.
- Apply a bounded workflow that uses relevant context and makes assumptions visible.
- Evaluate and verify AI-supported work before relying on it.

Start with the engineering decision

Select a tool by matching the work to its strengths and the data policy. The useful outcome is a decision or change that can be checked in the context where the software actually runs. AI can help you explore options and reduce drafting effort; it cannot supply missing requirements or take responsibility for the result.

This matters in modern ai developer workflow because plausible output can still conflict with project behavior, user expectations, or operational constraints. Begin with the observed problem and define what success looks like before requesting a solution. Keep assumptions visible so a teammate can challenge them.

A practical workflow

1. State the task in terms of the behavior or decision you need: select a tool by matching the work to its strengths and the data policy.
2. Supply only the relevant code, contract, evidence, and constraints; remove secrets and unrelated data.
3. Use ChatGPT for discussion and alternatives, an editor assistant for local completions, and a repository agent for bounded multi-file work.
4. Ask for assumptions and alternatives when the evidence is incomplete; do not let a confident tone substitute for a source or reproduction.
5. Review the result in the project, run proportionate checks, and keep, revise, or reject it based on observable evidence.

Worked scenario

You need to understand a reducer before changing it; first request a control-flow explanation, then make a small verified edit in the editor. A useful request separates what is known from what is suspected. For example: "Here is the sanitized behavior, relevant contract, and constraint. First identify the assumptions and propose the smallest experiment that distinguishes the likely causes. Do not change unrelated interfaces." This asks for reasoning that can be tested rather than a patch detached from evidence.

task-brief.txt

A compact brief; replace bracketed text with project-specific evidence.

Goal: Select a tool by matching the work to its strengths and the data policy.

Context: [relevant behavior, files, version, or sanitized evidence]

Constraints: [invariants, policy, compatibility, and out-of-scope work]

Request: Use ChatGPT for discussion and alternatives, an editor assistant for local completions, and a repository agent for bounded multi-file work.

Check: [test, source, reproduction, or review that will verify the result]

Review and verification

- Compare the suggestion with the canonical requirement and nearby project conventions.
- Check normal, boundary, and failure behavior; inspect what the proposed change leaves untouched.
- Run the narrowest useful automated check, then broader checks when the change crosses boundaries.
- Read the complete diff and explain the result in your own words before accepting it.

A plausible answer still needs evidence

A common failure is to treat a fluent answer as proof. Choosing the Right AI Tool requires context and judgment: verify claims against code, tests, authoritative documentation, or a reproducible observation. If the tool lacks necessary context, ask a question or gather evidence instead of filling the gap with a guess.

Use AI to make the next engineering decision easier to inspect; keep the decision and its verification yours.

Working principle

Practice

Practice: Choosing the Right AI Tool Optional

Choose one task and compare the required context, feedback loop, and verification for each tool.

1. Choose a small, permitted task related to Choosing the Right AI Tool.
2. Write the goal, relevant context, constraints, and observable acceptance criteria before consulting an AI tool.
3. Request a bounded explanation, plan, or draft; note assumptions that need confirmation.
4. Verify the result with a source, focused test, reproduction, or review checklist and record what you changed or rejected.

Constraints: Use synthetic or approved information only; never include credentials, private user data, or material you are not allowed to share.

Expected outcome: A short task brief, a reviewable AI contribution, and evidence that addresses: Choose one task and compare the required context, feedback loop, and verification for each tool.

Knowledge check

Select your answers before checking them in the course reader. The answer key is intentionally not included in this ebook.

Question 1 (required). A developer is working on Choosing the Right AI Tool and receives a confident AI recommendation. What is the strongest next step?

- A. Apply the workflow: Use ChatGPT for discussion and alternatives, an editor assistant for local completions, and a repository agent for bounded multi-file work. Then verify the outcome against project evidence.
- B. Accept the recommendation because it is detailed and sounds technically plausible.
- C. Ask the model to repeat the same answer more confidently, then proceed without a project check.

Question 2 (required). A teammate asks you to act on this task: Select a tool by matching the work to its strengths and the data policy. Some requirements or evidence are missing. What is the strongest response?

- A. Identify the missing assumption, gather the smallest useful evidence, narrow the request, and verify affected behavior.
- B. Let the AI infer the missing requirements and make a broad change so work continues quickly.
- C. Ignore the uncertainty because the requested change appears small.

Prompting for Software Development

Learning objectives

- Explain how Prompting for Software Development supports a software-development task.
- Apply a bounded workflow that uses relevant context and makes assumptions visible.
- Evaluate and verify AI-supported work before relying on it.

Start with the engineering decision

Turn a vague request into a task brief with observable success criteria. The useful outcome is a decision or change that can be checked in the context where the software actually runs. AI can help you explore options and reduce drafting effort; it cannot supply missing requirements or take responsibility for the result.

This matters in modern ai developer workflow because plausible output can still conflict with project behavior, user expectations, or operational constraints. Begin with the observed problem and define what success looks like before requesting a solution. Keep assumptions visible so a teammate can challenge them.

A practical workflow

1. State the task in terms of the behavior or decision you need: turn a vague request into a task brief with observable success criteria.
2. Supply only the relevant code, contract, evidence, and constraints; remove secrets and unrelated data.
3. State the goal, relevant files or behavior, constraints, desired response, and how correctness will be checked.
4. Ask for assumptions and alternatives when the evidence is incomplete; do not let a confident tone substitute for a source or reproduction.
5. Review the result in the project, run proportionate checks, and keep, revise, or reject it based on observable evidence.

Worked scenario

For a date parser bug, provide the accepted formats, current behavior, expected behavior, and a failing input; ask for a hypothesis and test before a patch. A useful request separates what is known from what is suspected. For example: “Here is the sanitized behavior, relevant contract, and constraint. First identify the assumptions and propose the smallest experiment that distinguishes the likely causes. Do not change unrelated interfaces.” This asks for reasoning that can be tested rather than a patch detached from evidence.

task-brief.txt

A compact brief; replace bracketed text with project-specific evidence.

Goal: Turn a vague request into a task brief with observable success criteria.

Context: [relevant behavior, files, version, or sanitized evidence]

Constraints: [invariants, policy, compatibility, and out-of-scope work]

Request: State the goal, relevant files or behavior, constraints, desired response, and how correctness will be checked.

Check: [test, source, reproduction, or review that will verify the result]

Review and verification

- Compare the suggestion with the canonical requirement and nearby project conventions.
- Check normal, boundary, and failure behavior; inspect what the proposed change leaves untouched.
- Run the narrowest useful automated check, then broader checks when the change crosses boundaries.
- Read the complete diff and explain the result in your own words before accepting it.

A plausible answer still needs evidence

A common failure is to treat a fluent answer as proof. Prompting for Software Development requires context and judgment: verify claims against code, tests, authoritative documentation, or a reproducible observation. If the tool lacks necessary context, ask a question or gather evidence instead of filling the gap with a guess.

Use AI to make the next engineering decision easier to inspect; keep the decision and its verification yours.

Working principle

Practice

Practice: Prompting for Software Development Optional

Rewrite a vague coding request as a bounded brief and name the evidence that would prove the result.

1. Choose a small, permitted task related to Prompting for Software Development.
2. Write the goal, relevant context, constraints, and observable acceptance criteria before consulting an AI tool.
3. Request a bounded explanation, plan, or draft; note assumptions that need confirmation.
4. Verify the result with a source, focused test, reproduction, or review checklist and record what you changed or rejected.

Constraints: Use synthetic or approved information only; never include credentials, private user data, or material you are not allowed to share.

Expected outcome: A short task brief, a reviewable AI contribution, and evidence that addresses: Rewrite a vague coding request as a bounded brief and name the evidence that would prove the result.

Knowledge check

Select your answers before checking them in the course reader. The answer key is intentionally not included in this ebook.

Question 1 (required). A developer is working on Prompting for Software Development and receives a confident AI recommendation. What is the strongest next step?

- A. Apply the workflow: State the goal, relevant files or behavior, constraints, desired response, and how correctness will be checked. Then verify the outcome against project evidence.
- B. Accept the recommendation because it is detailed and sounds technically plausible.
- C. Ask the model to repeat the same answer more confidently, then proceed without a project check.

Question 2 (required). A teammate asks you to act on this task: Turn a vague request into a task brief with observable success criteria. Some requirements or evidence are missing. What is the strongest response?

- A. Identify the missing assumption, gather the smallest useful evidence, narrow the request, and verify affected behavior.
- B. Let the AI infer the missing requirements and make a broad change so work continues quickly.
- C. Ignore the uncertainty because the requested change appears small.

When NOT to Use AI

Learning objectives

- Explain how When NOT to Use AI supports a software-development task.
- Apply a bounded workflow that uses relevant context and makes assumptions visible.
- Evaluate and verify AI-supported work before relying on it.

Start with the engineering decision

Recognize cases where privacy, uncertainty, or high consequence makes AI unsuitable or secondary. The useful outcome is a decision or change that can be checked in the context where the software actually runs. AI can help you explore options and reduce drafting effort; it cannot supply missing requirements or take responsibility for the result.

This matters in modern ai developer workflow because plausible output can still conflict with project behavior, user expectations, or operational constraints. Begin with the observed problem and define what success looks like before requesting a solution. Keep assumptions visible so a teammate can challenge them.

A practical workflow

1. State the task in terms of the behavior or decision you need: recognize cases where privacy, uncertainty, or high consequence makes AI unsuitable or secondary.
2. Supply only the relevant code, contract, evidence, and constraints; remove secrets and unrelated data.
3. Check policy and data sensitivity; use approved tools only; use deterministic tools and expert review for authoritative outcomes.
4. Ask for assumptions and alternatives when the evidence is incomplete; do not let a confident tone substitute for a source or reproduction.
5. Review the result in the project, run proportionate checks, and keep, revise, or reject it based on observable evidence.

Worked scenario

A production incident includes customer tokens in logs: redact them before any approved AI analysis and use incident procedures as the source of truth. A useful request separates what is known from what is suspected. For example: “Here is the sanitized behavior, relevant contract, and constraint. First identify the assumptions and propose the smallest experiment that distinguishes the likely causes. Do not change unrelated interfaces.” This asks for reasoning that can be tested rather than a patch detached from evidence.

task-brief.txt

A compact brief; replace bracketed text with project-specific evidence.

Goal: Recognize cases where privacy, uncertainty, or high consequence makes AI unsuitable or secondary.

Context: [relevant behavior, files, version, or sanitized evidence]

Constraints: [invariants, policy, compatibility, and out-of-scope work]

Request: Check policy and data sensitivity; use approved tools only; use deterministic tools and expert review for authoritative outcomes.

Check: [test, source, reproduction, or review that will verify the result]

Review and verification

- Compare the suggestion with the canonical requirement and nearby project conventions.
- Check normal, boundary, and failure behavior; inspect what the proposed change leaves untouched.
- Run the narrowest useful automated check, then broader checks when the change crosses boundaries.
- Read the complete diff and explain the result in your own words before accepting it.

A plausible answer still needs evidence

A common failure is to treat a fluent answer as proof. When NOT to Use AI requires context and judgment: verify claims against code, tests, authoritative documentation, or a reproducible observation. If the tool lacks necessary context, ask a question or gather evidence instead of filling the gap with a guess.

Use AI to make the next engineering decision easier to inspect; keep the decision and its verification yours.

Working principle

Practice

Practice: When NOT to Use AI Optional

Classify three tasks by risk and identify a safe alternative or required review for each.

1. Choose a small, permitted task related to When NOT to Use AI.
2. Write the goal, relevant context, constraints, and observable acceptance criteria before consulting an AI tool.
3. Request a bounded explanation, plan, or draft; note assumptions that need confirmation.
4. Verify the result with a source, focused test, reproduction, or review checklist and record what you changed or rejected.

Constraints: Use synthetic or approved information only; never include credentials, private user data, or material you are not allowed to share.

Expected outcome: A short task brief, a reviewable AI contribution, and evidence that addresses: Classify three tasks by risk and identify a safe alternative or required review for each.

Knowledge check

Select your answers before checking them in the course reader. The answer key is intentionally not included in this ebook.

Question 1 (required). A developer is working on When NOT to Use AI and receives a confident AI recommendation. What is the strongest next step?

- A. Apply the workflow: Check policy and data sensitivity; use approved tools only; use deterministic tools and expert review for authoritative outcomes. Then verify the outcome against project evidence.
- B. Accept the recommendation because it is detailed and sounds technically plausible.
- C. Ask the model to repeat the same answer more confidently, then proceed without a project check.

Question 2 (required). A teammate asks you to act on this task: Recognize cases where privacy, uncertainty, or high consequence makes AI unsuitable or secondary. Some requirements or evidence are missing. What is the strongest response?

- A. Identify the missing assumption, gather the smallest useful evidence, narrow the request, and verify affected behavior.
- B. Let the AI infer the missing requirements and make a broad change so work continues quickly.
- C. Ignore the uncertainty because the requested change appears small.

ChatGPT as a Development Assistant

Learning objectives

- Explain how ChatGPT as a Development Assistant supports a software-development task.
- Apply a bounded workflow that uses relevant context and makes assumptions visible.
- Evaluate and verify AI-supported work before relying on it.

Start with the engineering decision

Use a conversational assistant to explain, plan, draft, and critique work while retaining control. The useful outcome is a decision or change that can be checked in the context where the software actually runs. AI can help you explore options and reduce drafting effort; it cannot supply missing requirements or take responsibility for the result.

This matters in chatgpt for developers because plausible output can still conflict with project behavior, user expectations, or operational constraints. Begin with the observed problem and define what success looks like before requesting a solution. Keep assumptions visible so a teammate can challenge them.

A practical workflow

1. State the task in terms of the behavior or decision you need: use a conversational assistant to explain, plan, draft, and critique work while retaining control.
2. Supply only the relevant code, contract, evidence, and constraints; remove secrets and unrelated data.
3. Provide a small relevant excerpt and goal; ask for structured output; challenge assumptions; apply only a reviewed result.
4. Ask for assumptions and alternatives when the evidence is incomplete; do not let a confident tone substitute for a source or reproduction.
5. Review the result in the project, run proportionate checks, and keep, revise, or reject it based on observable evidence.

Worked scenario

Ask for a comparison of two implementations against stated constraints, then verify the recommendation against repository conventions and tests. A useful request separates what is known from what is suspected. For example: “Here is the sanitized behavior, relevant contract, and constraint. First identify the assumptions and propose the smallest experiment that distinguishes the likely causes. Do not change unrelated interfaces.” This asks for reasoning that can be tested rather than a patch detached from evidence.

task-brief.txt

A compact brief; replace bracketed text with project-specific evidence.

Goal: Use a conversational assistant to explain, plan, draft, and critique work while retaining control.

Context: [relevant behavior, files, version, or sanitized evidence]

Constraints: [invariants, policy, compatibility, and out-of-scope work]

Request: Provide a small relevant excerpt and goal; ask for structured output; challenge assumptions; apply only a reviewed result.

Check: [test, source, reproduction, or review that will verify the result]

Review and verification

- Compare the suggestion with the canonical requirement and nearby project conventions.
- Check normal, boundary, and failure behavior; inspect what the proposed change leaves untouched.
- Run the narrowest useful automated check, then broader checks when the change crosses boundaries.
- Read the complete diff and explain the result in your own words before accepting it.

A plausible answer still needs evidence

A common failure is to treat a fluent answer as proof. ChatGPT as a Development Assistant requires context and judgment: verify claims against code, tests, authoritative documentation, or a reproducible observation. If the tool lacks necessary context, ask a question or gather evidence instead of filling the gap with a guess.

Use AI to make the next engineering decision easier to inspect; keep the decision and its verification yours.

Working principle

Practice

Practice: ChatGPT as a Development Assistant Optional

Prepare a prompt and follow-up sequence for one task, marking where you independently check each answer.

1. Choose a small, permitted task related to ChatGPT as a Development Assistant.
2. Write the goal, relevant context, constraints, and observable acceptance criteria before consulting an AI tool.
3. Request a bounded explanation, plan, or draft; note assumptions that need confirmation.
4. Verify the result with a source, focused test, reproduction, or review checklist and record what you changed or rejected.

Constraints: Use synthetic or approved information only; never include credentials, private user data, or material you are not allowed to share.

Expected outcome: A short task brief, a reviewable AI contribution, and evidence that addresses: Prepare a prompt and follow-up sequence for one task, marking where you independently check each answer.

Knowledge check

Select your answers before checking them in the course reader. The answer key is intentionally not included in this ebook.

Question 1 (required). A developer is working on ChatGPT as a Development Assistant and receives a confident AI recommendation. What is the strongest next step?

- A. Apply the workflow: Provide a small relevant excerpt and goal; ask for structured output; challenge assumptions; apply only a reviewed result. Then verify the outcome against project evidence.
- B. Accept the recommendation because it is detailed and sounds technically plausible.
- C. Ask the model to repeat the same answer more confidently, then proceed without a project check.

Question 2 (required). A teammate asks you to act on this task: Use a conversational assistant to explain, plan, draft, and critique work while retaining control. Some requirements or evidence are missing. What is the strongest response?

- A. Identify the missing assumption, gather the smallest useful evidence, narrow the request, and verify affected behavior.
- B. Let the AI infer the missing requirements and make a broad change so work continues quickly.
- C. Ignore the uncertainty because the requested change appears small.

Debugging with ChatGPT

Learning objectives

- Explain how Debugging with ChatGPT supports a software-development task.
- Apply a bounded workflow that uses relevant context and makes assumptions visible.
- Evaluate and verify AI-supported work before relying on it.

Start with the engineering decision

Use evidence to narrow a bug instead of asking for an unsupported fix. The useful outcome is a decision or change that can be checked in the context where the software actually runs. AI can help you explore options and reduce drafting effort; it cannot supply missing requirements or take responsibility for the result.

This matters in chatgpt for developers because plausible output can still conflict with project behavior, user expectations, or operational constraints. Begin with the observed problem and define what success looks like before requesting a solution. Keep assumptions visible so a teammate can challenge them.

A practical workflow

1. State the task in terms of the behavior or decision you need: use evidence to narrow a bug instead of asking for an unsupported fix.
2. Supply only the relevant code, contract, evidence, and constraints; remove secrets and unrelated data.
3. Share expected and actual behavior, sanitized error, minimal relevant code, environment, and recent changes; request ranked hypotheses and discriminating checks.
4. Ask for assumptions and alternatives when the evidence is incomplete; do not let a confident tone substitute for a source or reproduction.
5. Review the result in the project, run proportionate checks, and keep, revise, or reject it based on observable evidence.

Worked scenario

If a form submits twice, inspect the event path and network calls before changing button state; test the suspected path with one focused reproduction. A useful request separates what is known from what is suspected. For example: “Here is the sanitized behavior, relevant contract, and constraint. First identify the assumptions and propose the smallest experiment that distinguishes the likely causes. Do not change unrelated interfaces.” This asks for reasoning that can be tested rather than a patch detached from evidence.

task-brief.txt

A compact brief; replace bracketed text with project-specific evidence.

Goal: Use evidence to narrow a bug instead of asking for an unsupported fix.

Context: [relevant behavior, files, version, or sanitized evidence]

Constraints: [invariants, policy, compatibility, and out-of-scope work]

Request: Share expected and actual behavior, sanitized error, minimal relevant code, environment, and recent changes; request ranked hypotheses and discriminating checks.

Check: [test, source, reproduction, or review that will verify the result]

Review and verification

- Compare the suggestion with the canonical requirement and nearby project conventions.
- Check normal, boundary, and failure behavior; inspect what the proposed change leaves untouched.
- Run the narrowest useful automated check, then broader checks when the change crosses boundaries.
- Read the complete diff and explain the result in your own words before accepting it.

A plausible answer still needs evidence

A common failure is to treat a fluent answer as proof. Debugging with ChatGPT requires context and judgment: verify claims against code, tests, authoritative documentation, or a reproducible observation. If the tool lacks necessary context, ask a question or gather evidence instead of filling the gap with a guess.

Use AI to make the next engineering decision easier to inspect; keep the decision and its verification yours.

Working principle

Practice

Practice: Debugging with ChatGPT Optional

Write a debugging brief for a reproducible issue and propose a test that distinguishes two plausible causes.

1. Choose a small, permitted task related to Debugging with ChatGPT.
2. Write the goal, relevant context, constraints, and observable acceptance criteria before consulting an AI tool.
3. Request a bounded explanation, plan, or draft; note assumptions that need confirmation.
4. Verify the result with a source, focused test, reproduction, or review checklist and record what you changed or rejected.

Constraints: Use synthetic or approved information only; never include credentials, private user data, or material you are not allowed to share.

Expected outcome: A short task brief, a reviewable AI contribution, and evidence that addresses: Write a debugging brief for a reproducible issue and propose a test that distinguishes two plausible causes.

Knowledge check

Select your answers before checking them in the course reader. The answer key is intentionally not included in this ebook.

Question 1 (required). A developer is working on Debugging with ChatGPT and receives a confident AI recommendation. What is the strongest next step?

- A. Apply the workflow: Share expected and actual behavior, sanitized error, minimal relevant code, environment, and recent changes; request ranked hypotheses and discriminating checks. Then verify the outcome against project evidence.
- B. Accept the recommendation because it is detailed and sounds technically plausible.
- C. Ask the model to repeat the same answer more confidently, then proceed without a project check.

Question 2 (required). A teammate asks you to act on this task: Use evidence to narrow a bug instead of asking for an unsupported fix. Some requirements or evidence are missing. What is the strongest response?

- A. Identify the missing assumption, gather the smallest useful evidence, narrow the request, and verify affected behavior.
- B. Let the AI infer the missing requirements and make a broad change so work continues quickly.
- C. Ignore the uncertainty because the requested change appears small.

Refactoring and Improving Existing Code

Learning objectives

- Explain how Refactoring and Improving Existing Code supports a software-development task.
- Apply a bounded workflow that uses relevant context and makes assumptions visible.
- Evaluate and verify AI-supported work before relying on it.

Start with the engineering decision

Improve structure while preserving observable behavior. The useful outcome is a decision or change that can be checked in the context where the software actually runs. AI can help you explore options and reduce drafting effort; it cannot supply missing requirements or take responsibility for the result.

This matters in chatgpt for developers because plausible output can still conflict with project behavior, user expectations, or operational constraints. Begin with the observed problem and define what success looks like before requesting a solution. Keep assumptions visible so a teammate can challenge them.

A practical workflow

1. State the task in terms of the behavior or decision you need: improve structure while preserving observable behavior.
2. Supply only the relevant code, contract, evidence, and constraints; remove secrets and unrelated data.
3. Define behavior and constraints first; request one small refactor; compare before and after; run focused tests and inspect the diff.
4. Ask for assumptions and alternatives when the evidence is incomplete; do not let a confident tone substitute for a source or reproduction.
5. Review the result in the project, run proportionate checks, and keep, revise, or reject it based on observable evidence.

Worked scenario

Extract repeated price formatting into a pure helper while preserving rounding, locale, and null behavior through characterization tests. A useful request separates what is known from what is suspected. For example: “Here is the sanitized behavior, relevant contract, and constraint. First identify the assumptions and propose the smallest experiment that distinguishes the likely causes. Do not change unrelated interfaces.” This asks for reasoning that can be tested rather than a patch detached from evidence.

task-brief.txt

A compact brief; replace bracketed text with project-specific evidence.

Goal: Improve structure while preserving observable behavior.

Context: [relevant behavior, files, version, or sanitized evidence]

Constraints: [invariants, policy, compatibility, and out-of-scope work]

Request: Define behavior and constraints first; request one small refactor; compare before and after; run focused tests and inspect the diff.

Check: [test, source, reproduction, or review that will verify the result]

Review and verification

- Compare the suggestion with the canonical requirement and nearby project conventions.
- Check normal, boundary, and failure behavior; inspect what the proposed change leaves untouched.
- Run the narrowest useful automated check, then broader checks when the change crosses boundaries.
- Read the complete diff and explain the result in your own words before accepting it.

A plausible answer still needs evidence

A common failure is to treat a fluent answer as proof. Refactoring and Improving Existing Code requires context and judgment: verify claims against code, tests, authoritative documentation, or a reproducible observation. If the tool lacks necessary context, ask a question or gather evidence instead of filling the gap with a guess.

Use AI to make the next engineering decision easier to inspect; keep the decision and its verification yours.

Working principle

Practice

Practice: Refactoring and Improving Existing Code Optional

Select a small refactor, record current behavior, and specify regression checks before changing structure.

1. Choose a small, permitted task related to Refactoring and Improving Existing Code.
2. Write the goal, relevant context, constraints, and observable acceptance criteria before consulting an AI tool.
3. Request a bounded explanation, plan, or draft; note assumptions that need confirmation.
4. Verify the result with a source, focused test, reproduction, or review checklist and record what you changed or rejected.

Constraints: Use synthetic or approved information only; never include credentials, private user data, or material you are not allowed to share.

Expected outcome: A short task brief, a reviewable AI contribution, and evidence that addresses: Select a small refactor, record current behavior, and specify regression checks before changing structure.

Knowledge check

Select your answers before checking them in the course reader. The answer key is intentionally not included in this ebook.

Question 1 (required). A developer is working on Refactoring and Improving Existing Code and receives a confident AI recommendation. What is the strongest next step?

- A. Apply the workflow: Define behavior and constraints first; request one small refactor; compare before and after; run focused tests and inspect the diff. Then verify the outcome against project evidence.
- B. Accept the recommendation because it is detailed and sounds technically plausible.
- C. Ask the model to repeat the same answer more confidently, then proceed without a project check.

Question 2 (required). A teammate asks you to act on this task: Improve structure while preserving observable behavior. Some requirements or evidence are missing. What is the strongest response?

- A. Identify the missing assumption, gather the smallest useful evidence, narrow the request, and verify affected behavior.
- B. Let the AI infer the missing requirements and make a broad change so work continues quickly.
- C. Ignore the uncertainty because the requested change appears small.

Frameworks, Libraries, and Documentation

Learning objectives

- Explain how Frameworks, Libraries, and Documentation supports a software-development task.
- Apply a bounded workflow that uses relevant context and makes assumptions visible.
- Evaluate and verify AI-supported work before relying on it.

Start with the engineering decision

Use current primary documentation to resolve framework and library questions. The useful outcome is a decision or change that can be checked in the context where the software actually runs. AI can help you explore options and reduce drafting effort; it cannot supply missing requirements or take responsibility for the result.

This matters in chatgpt for developers because plausible output can still conflict with project behavior, user expectations, or operational constraints. Begin with the observed problem and define what success looks like before requesting a solution. Keep assumptions visible so a teammate can challenge them.

A practical workflow

1. State the task in terms of the behavior or decision you need: use current primary documentation to resolve framework and library questions.
2. Supply only the relevant code, contract, evidence, and constraints; remove secrets and unrelated data.
3. Identify package and version; search official documentation or repository types; ask AI to explain a supplied passage; validate an example locally.
4. Ask for assumptions and alternatives when the evidence is incomplete; do not let a confident tone substitute for a source or reproduction.
5. Review the result in the project, run proportionate checks, and keep, revise, or reject it based on observable evidence.

Worked scenario

For a React hook question, include the installed React version and relevant component; compare the response with the official API reference. A useful request separates what is known from what is suspected. For example: “Here is the sanitized behavior, relevant contract, and constraint. First identify the assumptions and propose the smallest experiment that distinguishes the likely causes. Do not change unrelated interfaces.” This asks for reasoning that can be tested rather than a patch detached from evidence.

task-brief.txt

A compact brief; replace bracketed text with project-specific evidence.

Goal: Use current primary documentation to resolve framework and library questions.

Context: [relevant behavior, files, version, or sanitized evidence]

Constraints: [invariants, policy, compatibility, and out-of-scope work]

Request: Identify package and version; search official documentation or repository types; ask AI to explain a supplied passage; validate an example locally.

Check: [test, source, reproduction, or review that will verify the result]

Review and verification

- Compare the suggestion with the canonical requirement and nearby project conventions.
- Check normal, boundary, and failure behavior; inspect what the proposed change leaves untouched.
- Run the narrowest useful automated check, then broader checks when the change crosses boundaries.
- Read the complete diff and explain the result in your own words before accepting it.

A plausible answer still needs evidence

A common failure is to treat a fluent answer as proof. Frameworks, Libraries, and Documentation requires context and judgment: verify claims against code, tests, authoritative documentation, or a reproducible observation. If the tool lacks necessary context, ask a question or gather evidence instead of filling the gap with a guess.

Use AI to make the next engineering decision easier to inspect; keep the decision and its verification yours.

Working principle

Practice

Practice: Frameworks, Libraries, and Documentation Optional

Investigate one version-sensitive API and record the version, authoritative source, and runnable verification.

1. Choose a small, permitted task related to Frameworks, Libraries, and Documentation.
2. Write the goal, relevant context, constraints, and observable acceptance criteria before consulting an AI tool.
3. Request a bounded explanation, plan, or draft; note assumptions that need confirmation.
4. Verify the result with a source, focused test, reproduction, or review checklist and record what you changed or rejected.

Constraints: Use synthetic or approved information only; never include credentials, private user data, or material you are not allowed to share.

Expected outcome: A short task brief, a reviewable AI contribution, and evidence that addresses: Investigate one version-sensitive API and record the version, authoritative source, and runnable verification.

Knowledge check

Select your answers before checking them in the course reader. The answer key is intentionally not included in this ebook.

Question 1 (required). A developer is working on Frameworks, Libraries, and Documentation and receives a confident AI recommendation. What is the strongest next step?

- A. Apply the workflow: Identify package and version; search official documentation or repository types; ask AI to explain a supplied passage; validate an example locally. Then verify the outcome against project evidence.
- B. Accept the recommendation because it is detailed and sounds technically plausible.
- C. Ask the model to repeat the same answer more confidently, then proceed without a project check.

Question 2 (required). A teammate asks you to act on this task: Use current primary documentation to resolve framework and library questions Some requirements or evidence are missing. What is the strongest response?

- A. Identify the missing assumption, gather the smallest useful evidence, narrow the request, and verify affected behavior.
- B. Let the AI infer the missing requirements and make a broad change so work continues quickly.
- C. Ignore the uncertainty because the requested change appears small.

Understanding Errors and Stack Traces

Learning objectives

- Explain how Understanding Errors and Stack Traces supports a software-development task.
- Apply a bounded workflow that uses relevant context and makes assumptions visible.
- Evaluate and verify AI-supported work before relying on it.

Start with the engineering decision

Read an error trace from the failure site outward and map frames to your code. The useful outcome is a decision or change that can be checked in the context where the software actually runs. AI can help you explore options and reduce drafting effort; it cannot supply missing requirements or take responsibility for the result.

This matters in chatgpt for developers because plausible output can still conflict with project behavior, user expectations, or operational constraints. Begin with the observed problem and define what success looks like before requesting a solution. Keep assumptions visible so a teammate can challenge them.

A practical workflow

1. State the task in terms of the behavior or decision you need: read an error trace from the failure site outward and map frames to your code.
2. Supply only the relevant code, contract, evidence, and constraints; remove secrets and unrelated data.
3. Separate runtime frames from project frames; inspect the first relevant call site; trace values and reproduce with a small input.
4. Ask for assumptions and alternatives when the evidence is incomplete; do not let a confident tone substitute for a source or reproduction.
5. Review the result in the project, run proportionate checks, and keep, revise, or reject it based on observable evidence.

Worked scenario

A `TypeError` in a route handler may originate from an absent nested field; follow the project frame to the request parsing boundary before adding a guard. A useful request separates what is known from what is suspected. For example: “Here is the sanitized behavior, relevant contract, and constraint. First identify the assumptions and propose the smallest experiment that distinguishes the likely causes. Do not change unrelated interfaces.” This asks for reasoning that can be tested rather than a patch detached from evidence.

task-brief.txt

A compact brief; replace bracketed text with project-specific evidence.

Goal: Read an error trace from the failure site outward and map frames to your code.

Context: [relevant behavior, files, version, or sanitized evidence]

Constraints: [invariants, policy, compatibility, and out-of-scope work]

Request: Separate runtime frames from project frames; inspect the first relevant call site; trace values and reproduce with a small input.

Check: [test, source, reproduction, or review that will verify the result]

Review and verification

- Compare the suggestion with the canonical requirement and nearby project conventions.
- Check normal, boundary, and failure behavior; inspect what the proposed change leaves untouched.
- Run the narrowest useful automated check, then broader checks when the change crosses boundaries.
- Read the complete diff and explain the result in your own words before accepting it.

A plausible answer still needs evidence

A common failure is to treat a fluent answer as proof. Understanding Errors and Stack Traces requires context and judgment: verify claims against code, tests, authoritative documentation, or a reproducible observation. If the tool lacks necessary context, ask a question or gather evidence instead of filling the gap with a guess.

Use AI to make the next engineering decision easier to inspect; keep the decision and its verification yours.

Working principle

Practice

Practice: Understanding Errors and Stack Traces Optional

Annotate a sanitized trace with the first project frame, suspected input, and a confirming experiment.

1. Choose a small, permitted task related to Understanding Errors and Stack Traces.
2. Write the goal, relevant context, constraints, and observable acceptance criteria before consulting an AI tool.
3. Request a bounded explanation, plan, or draft; note assumptions that need confirmation.
4. Verify the result with a source, focused test, reproduction, or review checklist and record what you changed or rejected.

Constraints: Use synthetic or approved information only; never include credentials, private user data, or material you are not allowed to share.

Expected outcome: A short task brief, a reviewable AI contribution, and evidence that addresses:
Annotate a sanitized trace with the first project frame, suspected input, and a confirming experiment.

Knowledge check

Select your answers before checking them in the course reader. The answer key is intentionally not included in this ebook.

Question 1 (required). A developer is working on Understanding Errors and Stack Traces and receives a confident AI recommendation. What is the strongest next step?

- A. Apply the workflow: Separate runtime frames from project frames; inspect the first relevant call site; trace values and reproduce with a small input. Then verify the outcome against project evidence.
- B. Accept the recommendation because it is detailed and sounds technically plausible.
- C. Ask the model to repeat the same answer more confidently, then proceed without a project check.

Question 2 (required). A teammate asks you to act on this task: Read an error trace from the failure site outward and map frames to your code. Some requirements or evidence are missing. What is the strongest response?

- A. Identify the missing assumption, gather the smallest useful evidence, narrow the request, and verify affected behavior.
- B. Let the AI infer the missing requirements and make a broad change so work continues quickly.
- C. Ignore the uncertainty because the requested change appears small.

APIs, SQL, and Development Questions

Learning objectives

- Explain how APIs, SQL, and Development Questions supports a software-development task.
- Apply a bounded workflow that uses relevant context and makes assumptions visible.
- Evaluate and verify AI-supported work before relying on it.

Start with the engineering decision

Ask precise questions about interfaces and data behavior, then verify semantics. The useful outcome is a decision or change that can be checked in the context where the software actually runs. AI can help you explore options and reduce drafting effort; it cannot supply missing requirements or take responsibility for the result.

This matters in chatgpt for developers because plausible output can still conflict with project behavior, user expectations, or operational constraints. Begin with the observed problem and define what success looks like before requesting a solution. Keep assumptions visible so a teammate can challenge them.

A practical workflow

1. State the task in terms of the behavior or decision you need: ask precise questions about interfaces and data behavior, then verify semantics.
2. Supply only the relevant code, contract, evidence, and constraints; remove secrets and unrelated data.
3. Include method, route or query, schema, expected result, constraints, and relevant version; test edge cases and parameterization.
4. Ask for assumptions and alternatives when the evidence is incomplete; do not let a confident tone substitute for a source or reproduction.
5. Review the result in the project, run proportionate checks, and keep, revise, or reject it based on observable evidence.

Worked scenario

For a paginated API, state ordering and cursor semantics; check empty pages and duplicate sort values rather than trusting an illustrative query. A useful request separates what is known from what is suspected. For example: “Here is the sanitized behavior, relevant contract, and constraint. First identify the assumptions and propose the smallest experiment that distinguishes the likely causes. Do not change unrelated interfaces.” This asks for reasoning that can be tested rather than a patch detached from evidence.

task-brief.txt

A compact brief; replace bracketed text with project-specific evidence.

Goal: Ask precise questions about interfaces and data behavior, then verify semantics.

Context: [relevant behavior, files, version, or sanitized evidence]

Constraints: [invariants, policy, compatibility, and out-of-scope work]

Request: Include method, route or query, schema, expected result, constraints, and relevant version; test edge cases and parameterization.

Check: [test, source, reproduction, or review that will verify the result]

Review and verification

- Compare the suggestion with the canonical requirement and nearby project conventions.
- Check normal, boundary, and failure behavior; inspect what the proposed change leaves untouched.
- Run the narrowest useful automated check, then broader checks when the change crosses boundaries.
- Read the complete diff and explain the result in your own words before accepting it.

A plausible answer still needs evidence

A common failure is to treat a fluent answer as proof. APIs, SQL, and Development Questions requires context and judgment: verify claims against code, tests, authoritative documentation, or a reproducible observation. If the tool lacks necessary context, ask a question or gather evidence instead of filling the gap with a guess.

Use AI to make the next engineering decision easier to inspect; keep the decision and its verification yours.

Working principle

Practice

Practice: APIs, SQL, and Development Questions Optional

Draft a question about an API or SQL behavior and list normal, boundary, and invalid cases to verify.

1. Choose a small, permitted task related to APIs, SQL, and Development Questions.
2. Write the goal, relevant context, constraints, and observable acceptance criteria before consulting an AI tool.
3. Request a bounded explanation, plan, or draft; note assumptions that need confirmation.
4. Verify the result with a source, focused test, reproduction, or review checklist and record what you changed or rejected.

Constraints: Use synthetic or approved information only; never include credentials, private user data, or material you are not allowed to share.

Expected outcome: A short task brief, a reviewable AI contribution, and evidence that addresses: Draft a question about an API or SQL behavior and list normal, boundary, and invalid cases to verify.

Knowledge check

Select your answers before checking them in the course reader. The answer key is intentionally not included in this ebook.

Question 1 (required). A developer is working on APIs, SQL, and Development Questions and receives a confident AI recommendation. What is the strongest next step?

- A. Apply the workflow: Include method, route or query, schema, expected result, constraints, and relevant version; test edge cases and parameterization. Then verify the outcome against project evidence.
- B. Accept the recommendation because it is detailed and sounds technically plausible.
- C. Ask the model to repeat the same answer more confidently, then proceed without a project check.

Question 2 (required). A teammate asks you to act on this task: Ask precise questions about interfaces and data behavior, then verify semantics. Some requirements or evidence are missing. What is the strongest response?

- A. Identify the missing assumption, gather the smallest useful evidence, narrow the request, and verify affected behavior.
- B. Let the AI infer the missing requirements and make a broad change so work continues quickly.
- C. Ignore the uncertainty because the requested change appears small.

Architecture and Technical Reasoning

Learning objectives

- Explain how Architecture and Technical Reasoning supports a software-development task.
- Apply a bounded workflow that uses relevant context and makes assumptions visible.
- Evaluate and verify AI-supported work before relying on it.

Start with the engineering decision

Use AI to expose trade-offs and assumptions without outsourcing architecture decisions. The useful outcome is a decision or change that can be checked in the context where the software actually runs. AI can help you explore options and reduce drafting effort; it cannot supply missing requirements or take responsibility for the result.

This matters in chatgpt for developers because plausible output can still conflict with project behavior, user expectations, or operational constraints. Begin with the observed problem and define what success looks like before requesting a solution. Keep assumptions visible so a teammate can challenge them.

A practical workflow

1. State the task in terms of the behavior or decision you need: use AI to expose trade-offs and assumptions without outsourcing architecture decisions.
2. Supply only the relevant code, contract, evidence, and constraints; remove secrets and unrelated data.
3. Describe requirements, scale assumptions, constraints, and existing boundaries; compare options against explicit criteria; record the decision and revisit triggers.
4. Ask for assumptions and alternatives when the evidence is incomplete; do not let a confident tone substitute for a source or reproduction.
5. Review the result in the project, run proportionate checks, and keep, revise, or reject it based on observable evidence.

Worked scenario

Compare a synchronous request with a background job for report generation using latency, retry, failure visibility, and operational complexity. A useful request separates what is known from what is suspected. For example: “Here is the sanitized behavior, relevant contract, and constraint. First identify the assumptions and propose the smallest experiment that distinguishes the likely causes. Do not change unrelated interfaces.” This asks for reasoning that can be tested rather than a patch detached from evidence.

task-brief.txt

A compact brief; replace bracketed text with project-specific evidence.

Goal: Use AI to expose trade-offs and assumptions without outsourcing architecture decisions.

Context: [relevant behavior, files, version, or sanitized evidence]

Constraints: [invariants, policy, compatibility, and out-of-scope work]

Request: Describe requirements, scale assumptions, constraints, and existing boundaries; compare options against explicit criteria; record the decision and revisit triggers.

Check: [test, source, reproduction, or review that will verify the result]

Review and verification

- Compare the suggestion with the canonical requirement and nearby project conventions.
- Check normal, boundary, and failure behavior; inspect what the proposed change leaves untouched.
- Run the narrowest useful automated check, then broader checks when the change crosses boundaries.
- Read the complete diff and explain the result in your own words before accepting it.

A plausible answer still needs evidence

A common failure is to treat a fluent answer as proof. Architecture and Technical Reasoning requires context and judgment: verify claims against code, tests, authoritative documentation, or a reproducible observation. If the tool lacks necessary context, ask a question or gather evidence instead of filling the gap with a guess.

Use AI to make the next engineering decision easier to inspect; keep the decision and its verification yours.

Working principle

Practice

Practice: Architecture and Technical Reasoning Optional

Create a short decision record with two alternatives, evidence, trade-offs, and one assumption to validate.

1. Choose a small, permitted task related to Architecture and Technical Reasoning.
2. Write the goal, relevant context, constraints, and observable acceptance criteria before consulting an AI tool.
3. Request a bounded explanation, plan, or draft; note assumptions that need confirmation.
4. Verify the result with a source, focused test, reproduction, or review checklist and record what you changed or rejected.

Constraints: Use synthetic or approved information only; never include credentials, private user data, or material you are not allowed to share.

Expected outcome: A short task brief, a reviewable AI contribution, and evidence that addresses: Create a short decision record with two alternatives, evidence, trade-offs, and one assumption to validate.

Knowledge check

Select your answers before checking them in the course reader. The answer key is intentionally not included in this ebook.

Question 1 (required). A developer is working on Architecture and Technical Reasoning and receives a confident AI recommendation. What is the strongest next step?

- A. Apply the workflow: Describe requirements, scale assumptions, constraints, and existing boundaries; compare options against explicit criteria; record the decision and revisit triggers. Then verify the outcome against project evidence.
- B. Accept the recommendation because it is detailed and sounds technically plausible.
- C. Ask the model to repeat the same answer more confidently, then proceed without a project check.

Question 2 (required). A teammate asks you to act on this task: Use AI to expose trade-offs and assumptions without outsourcing architecture decisions. Some requirements or evidence are missing. What is the strongest response?

- A. Identify the missing assumption, gather the smallest useful evidence, narrow the request, and verify affected behavior.
- B. Let the AI infer the missing requirements and make a broad change so work continues quickly.
- C. Ignore the uncertainty because the requested change appears small.

Getting Started with GitHub Copilot

Learning objectives

- Explain how Getting Started with GitHub Copilot supports a software-development task.
- Apply a bounded workflow that uses relevant context and makes assumptions visible.
- Evaluate and verify AI-supported work before relying on it.

Start with the engineering decision

Establish an editor workflow that treats Copilot suggestions as optional drafts. The useful outcome is a decision or change that can be checked in the context where the software actually runs. AI can help you explore options and reduce drafting effort; it cannot supply missing requirements or take responsibility for the result.

This matters in github copilot because plausible output can still conflict with project behavior, user expectations, or operational constraints. Begin with the observed problem and define what success looks like before requesting a solution. Keep assumptions visible so a teammate can challenge them.

A practical workflow

1. State the task in terms of the behavior or decision you need: establish an editor workflow that treats Copilot suggestions as optional drafts.
2. Supply only the relevant code, contract, evidence, and constraints; remove secrets and unrelated data.
3. Orient to the file and nearby patterns; state intent in names or comments; inspect the complete suggestion before accepting and verify afterward.
4. Ask for assumptions and alternatives when the evidence is incomplete; do not let a confident tone substitute for a source or reproduction.
5. Review the result in the project, run proportionate checks, and keep, revise, or reject it based on observable evidence.

Worked scenario

In a repository with existing validation helpers, inspect nearby tests and naming before accepting a generated helper. A useful request separates what is known from what is suspected. For example: “Here is the sanitized behavior, relevant contract, and constraint. First identify the assumptions and propose the smallest experiment that distinguishes the likely causes. Do not change unrelated interfaces.” This asks for reasoning that can be tested rather than a patch detached from evidence.

task-brief.txt

A compact brief; replace bracketed text with project-specific evidence.

Goal: Establish an editor workflow that treats Copilot suggestions as optional drafts.

Context: [relevant behavior, files, version, or sanitized evidence]

Constraints: [invariants, policy, compatibility, and out-of-scope work]

Request: Orient to the file and nearby patterns; state intent in names or comments; inspect the complete suggestion before accepting and verify afterward.

Check: [test, source, reproduction, or review that will verify the result]

Review and verification

- Compare the suggestion with the canonical requirement and nearby project conventions.
- Check normal, boundary, and failure behavior; inspect what the proposed change leaves untouched.
- Run the narrowest useful automated check, then broader checks when the change crosses boundaries.
- Read the complete diff and explain the result in your own words before accepting it.

A plausible answer still needs evidence

A common failure is to treat a fluent answer as proof. Getting Started with GitHub Copilot requires context and judgment: verify claims against code, tests, authoritative documentation, or a reproducible observation. If the tool lacks necessary context, ask a question or gather evidence instead of filling the gap with a guess.

Use AI to make the next engineering decision easier to inspect; keep the decision and its verification yours.

Working principle

Practice

Practice: Getting Started with GitHub Copilot Optional

Choose an editor task and write the context, acceptance check, and reason you would reject a plausible suggestion.

1. Choose a small, permitted task related to Getting Started with GitHub Copilot.
2. Write the goal, relevant context, constraints, and observable acceptance criteria before consulting an AI tool.
3. Request a bounded explanation, plan, or draft; note assumptions that need confirmation.
4. Verify the result with a source, focused test, reproduction, or review checklist and record what you changed or rejected.

Constraints: Use synthetic or approved information only; never include credentials, private user data, or material you are not allowed to share.

Expected outcome: A short task brief, a reviewable AI contribution, and evidence that addresses: Choose an editor task and write the context, acceptance check, and reason you would reject a plausible suggestion.

Knowledge check

Select your answers before checking them in the course reader. The answer key is intentionally not included in this ebook.

Question 1 (required). A developer is working on Getting Started with GitHub Copilot and receives a confident AI recommendation. What is the strongest next step?

- A. Apply the workflow: Orient to the file and nearby patterns; state intent in names or comments; inspect the complete suggestion before accepting and verify afterward. Then verify the outcome against project evidence.
- B. Accept the recommendation because it is detailed and sounds technically plausible.
- C. Ask the model to repeat the same answer more confidently, then proceed without a project check.

Question 2 (required). A teammate asks you to act on this task: Establish an editor workflow that treats Copilot suggestions as optional drafts. Some requirements or evidence are missing. What is the strongest response?

- A. Identify the missing assumption, gather the smallest useful evidence, narrow the request, and verify affected behavior.
- B. Let the AI infer the missing requirements and make a broad change so work continues quickly.
- C. Ignore the uncertainty because the requested change appears small.

Intelligent Autocomplete

Learning objectives

- Explain how Intelligent Autocomplete supports a software-development task.
- Apply a bounded workflow that uses relevant context and makes assumptions visible.
- Evaluate and verify AI-supported work before relying on it.

Start with the engineering decision

Shape inline completions by making local intent and types clear. The useful outcome is a decision or change that can be checked in the context where the software actually runs. AI can help you explore options and reduce drafting effort; it cannot supply missing requirements or take responsibility for the result.

This matters in github copilot because plausible output can still conflict with project behavior, user expectations, or operational constraints. Begin with the observed problem and define what success looks like before requesting a solution. Keep assumptions visible so a teammate can challenge them.

A practical workflow

1. State the task in terms of the behavior or decision you need: shape inline completions by making local intent and types clear.
2. Supply only the relevant code, contract, evidence, and constraints; remove secrets and unrelated data.
3. Use descriptive names and nearby examples; request a small completion; read the full result for boundary behavior and hidden assumptions.
4. Ask for assumptions and alternatives when the evidence is incomplete; do not let a confident tone substitute for a source or reproduction.
5. Review the result in the project, run proportionate checks, and keep, revise, or reject it based on observable evidence.

Worked scenario

A function signature that accepts a parsed date and returns a result type gives better constraints than a vague `'handleData'` stub. A useful request separates what is known from what is suspected. For example: "Here is the sanitized behavior, relevant contract, and constraint. First identify the assumptions and propose the smallest experiment that distinguishes the likely causes. Do not change unrelated interfaces." This asks for reasoning that can be tested rather than a patch detached from evidence.

task-brief.txt

A compact brief; replace bracketed text with project-specific evidence.

Goal: Shape inline completions by making local intent and types clear.

Context: [relevant behavior, files, version, or sanitized evidence]

Constraints: [invariants, policy, compatibility, and out-of-scope work]

Request: Use descriptive names and nearby examples; request a small completion; read the full result for boundary behavior and hidden assumptions.

Check: [test, source, reproduction, or review that will verify the result]

Review and verification

- Compare the suggestion with the canonical requirement and nearby project conventions.
- Check normal, boundary, and failure behavior; inspect what the proposed change leaves untouched.
- Run the narrowest useful automated check, then broader checks when the change crosses boundaries.
- Read the complete diff and explain the result in your own words before accepting it.

A plausible answer still needs evidence

A common failure is to treat a fluent answer as proof. Intelligent Autocomplete requires context and judgment: verify claims against code, tests, authoritative documentation, or a reproducible observation. If the tool lacks necessary context, ask a question or gather evidence instead of filling the gap with a guess.

Use AI to make the next engineering decision easier to inspect; keep the decision and its verification yours.

Working principle

Practice

Practice: Intelligent Autocomplete Optional

Improve a small function signature or surrounding context and review two possible completions for correctness.

1. Choose a small, permitted task related to Intelligent Autocomplete.
2. Write the goal, relevant context, constraints, and observable acceptance criteria before consulting an AI tool.
3. Request a bounded explanation, plan, or draft; note assumptions that need confirmation.
4. Verify the result with a source, focused test, reproduction, or review checklist and record what you changed or rejected.

Constraints: Use synthetic or approved information only; never include credentials, private user data, or material you are not allowed to share.

Expected outcome: A short task brief, a reviewable AI contribution, and evidence that addresses: Improve a small function signature or surrounding context and review two possible completions for correctness.

Knowledge check

Select your answers before checking them in the course reader. The answer key is intentionally not included in this ebook.

Question 1 (required). A developer is working on Intelligent Autocomplete and receives a confident AI recommendation. What is the strongest next step?

- A. Apply the workflow: Use descriptive names and nearby examples; request a small completion; read the full result for boundary behavior and hidden assumptions. Then verify the outcome against project evidence.
- B. Accept the recommendation because it is detailed and sounds technically plausible.
- C. Ask the model to repeat the same answer more confidently, then proceed without a project check.

Question 2 (required). A teammate asks you to act on this task: Shape inline completions by making local intent and types clear. Some requirements or evidence are missing. What is the strongest response?

- A. Identify the missing assumption, gather the smallest useful evidence, narrow the request, and verify affected behavior.
- B. Let the AI infer the missing requirements and make a broad change so work continues quickly.
- C. Ignore the uncertainty because the requested change appears small.

Generating Code with Copilot

Learning objectives

- Explain how Generating Code with Copilot supports a software-development task.
- Apply a bounded workflow that uses relevant context and makes assumptions visible.
- Evaluate and verify AI-supported work before relying on it.

Start with the engineering decision

Generate a bounded implementation that follows local conventions. The useful outcome is a decision or change that can be checked in the context where the software actually runs. AI can help you explore options and reduce drafting effort; it cannot supply missing requirements or take responsibility for the result.

This matters in github copilot because plausible output can still conflict with project behavior, user expectations, or operational constraints. Begin with the observed problem and define what success looks like before requesting a solution. Keep assumptions visible so a teammate can challenge them.

A practical workflow

1. State the task in terms of the behavior or decision you need: generate a bounded implementation that follows local conventions.
2. Supply only the relevant code, contract, evidence, and constraints; remove secrets and unrelated data.
3. Inspect similar code; provide types, input/output behavior, and edge cases; accept a narrow proposal; inspect diff and run checks.
4. Ask for assumptions and alternatives when the evidence is incomplete; do not let a confident tone substitute for a source or reproduction.
5. Review the result in the project, run proportionate checks, and keep, revise, or reject it based on observable evidence.

Worked scenario

For an Express handler, specify validation, status codes, error conventions, and whether persistence belongs in a service. A useful request separates what is known from what is suspected. For example: “Here is the sanitized behavior, relevant contract, and constraint. First identify the assumptions and propose the smallest experiment that distinguishes the likely causes. Do not change unrelated interfaces.” This asks for reasoning that can be tested rather than a patch detached from evidence.

task-brief.txt

A compact brief; replace bracketed text with project-specific evidence.

Goal: Generate a bounded implementation that follows local conventions.

Context: [relevant behavior, files, version, or sanitized evidence]

Constraints: [invariants, policy, compatibility, and out-of-scope work]

Request: Inspect similar code; provide types, input/output behavior, and edge cases; accept a narrow proposal; inspect diff and run checks.

Check: [test, source, reproduction, or review that will verify the result]

Review and verification

- Compare the suggestion with the canonical requirement and nearby project conventions.
- Check normal, boundary, and failure behavior; inspect what the proposed change leaves untouched.
- Run the narrowest useful automated check, then broader checks when the change crosses boundaries.
- Read the complete diff and explain the result in your own words before accepting it.

A plausible answer still needs evidence

A common failure is to treat a fluent answer as proof. Generating Code with Copilot requires context and judgment: verify claims against code, tests, authoritative documentation, or a reproducible observation. If the tool lacks necessary context, ask a question or gather evidence instead of filling the gap with a guess.

Use AI to make the next engineering decision easier to inspect; keep the decision and its verification yours.

Working principle

Practice

Practice: Generating Code with Copilot Optional

Implement a small feature behind an explicit acceptance checklist; record every assumption you had to resolve.

1. Choose a small, permitted task related to Generating Code with Copilot.
2. Write the goal, relevant context, constraints, and observable acceptance criteria before consulting an AI tool.
3. Request a bounded explanation, plan, or draft; note assumptions that need confirmation.
4. Verify the result with a source, focused test, reproduction, or review checklist and record what you changed or rejected.

Constraints: Use synthetic or approved information only; never include credentials, private user data, or material you are not allowed to share.

Expected outcome: A short task brief, a reviewable AI contribution, and evidence that addresses: Implement a small feature behind an explicit acceptance checklist; record every assumption you had to resolve.

Knowledge check

Select your answers before checking them in the course reader. The answer key is intentionally not included in this ebook.

Question 1 (required). A developer is working on Generating Code with Copilot and receives a confident AI recommendation. What is the strongest next step?

- A. Apply the workflow: Inspect similar code; provide types, input/output behavior, and edge cases; accept a narrow proposal; inspect diff and run checks. Then verify the outcome against project evidence.
- B. Accept the recommendation because it is detailed and sounds technically plausible.
- C. Ask the model to repeat the same answer more confidently, then proceed without a project check.

Question 2 (required). A teammate asks you to act on this task: Generate a bounded implementation that follows local conventions. Some requirements or evidence are missing. What is the strongest response?

- A. Identify the missing assumption, gather the smallest useful evidence, narrow the request, and verify affected behavior.
- B. Let the AI infer the missing requirements and make a broad change so work continues quickly.
- C. Ignore the uncertainty because the requested change appears small.

Writing Tests with Copilot

Learning objectives

- Explain how Writing Tests with Copilot supports a software-development task.
- Apply a bounded workflow that uses relevant context and makes assumptions visible.
- Evaluate and verify AI-supported work before relying on it.

Start with the engineering decision

Use generated tests to explore behavior, while ensuring assertions prove requirements. The useful outcome is a decision or change that can be checked in the context where the software actually runs. AI can help you explore options and reduce drafting effort; it cannot supply missing requirements or take responsibility for the result.

This matters in github copilot because plausible output can still conflict with project behavior, user expectations, or operational constraints. Begin with the observed problem and define what success looks like before requesting a solution. Keep assumptions visible so a teammate can challenge them.

A practical workflow

1. State the task in terms of the behavior or decision you need: use generated tests to explore behavior, while ensuring assertions prove requirements.
2. Supply only the relevant code, contract, evidence, and constraints; remove secrets and unrelated data.
3. Give the function contract and test framework; request cases including boundaries; check each assertion and run tests against both normal and faulty implementations.
4. Ask for assumptions and alternatives when the evidence is incomplete; do not let a confident tone substitute for a source or reproduction.
5. Review the result in the project, run proportionate checks, and keep, revise, or reject it based on observable evidence.

Worked scenario

A test for an access check should fail when ownership is removed; a test that only asserts status 200 may miss the security requirement. A useful request separates what is known from what is suspected. For example: “Here is the sanitized behavior, relevant contract, and constraint. First identify the assumptions and propose the smallest experiment that distinguishes the likely causes. Do not change unrelated interfaces.” This asks for reasoning that can be tested rather than a patch detached from evidence.

task-brief.txt

A compact brief; replace bracketed text with project-specific evidence.

Goal: Use generated tests to explore behavior, while ensuring assertions prove requirements.

Context: [relevant behavior, files, version, or sanitized evidence]

Constraints: [invariants, policy, compatibility, and out-of-scope work]

Request: Give the function contract and test framework; request cases including boundaries; check each assertion and run tests against both normal and faulty implementations.

Check: [test, source, reproduction, or review that will verify the result]

Review and verification

- Compare the suggestion with the canonical requirement and nearby project conventions.
- Check normal, boundary, and failure behavior; inspect what the proposed change leaves untouched.
- Run the narrowest useful automated check, then broader checks when the change crosses boundaries.
- Read the complete diff and explain the result in your own words before accepting it.

A plausible answer still needs evidence

A common failure is to treat a fluent answer as proof. Writing Tests with Copilot requires context and judgment: verify claims against code, tests, authoritative documentation, or a reproducible observation. If the tool lacks necessary context, ask a question or gather evidence instead of filling the gap with a guess.

Use AI to make the next engineering decision easier to inspect; keep the decision and its verification yours.

Working principle

Practice

Practice: Writing Tests with Copilot Optional

Ask for a test matrix for a function and assess whether each test would catch a realistic regression.

1. Choose a small, permitted task related to Writing Tests with Copilot.
2. Write the goal, relevant context, constraints, and observable acceptance criteria before consulting an AI tool.
3. Request a bounded explanation, plan, or draft; note assumptions that need confirmation.
4. Verify the result with a source, focused test, reproduction, or review checklist and record what you changed or rejected.

Constraints: Use synthetic or approved information only; never include credentials, private user data, or material you are not allowed to share.

Expected outcome: A short task brief, a reviewable AI contribution, and evidence that addresses: Ask for a test matrix for a function and assess whether each test would catch a realistic regression.

Knowledge check

Select your answers before checking them in the course reader. The answer key is intentionally not included in this ebook.

Question 1 (required). A developer is working on Writing Tests with Copilot and receives a confident AI recommendation. What is the strongest next step?

- A. Apply the workflow: Give the function contract and test framework; request cases including boundaries; check each assertion and run tests against both normal and faulty implementations. Then verify the outcome against project evidence.
- B. Accept the recommendation because it is detailed and sounds technically plausible.
- C. Ask the model to repeat the same answer more confidently, then proceed without a project check.

Question 2 (required). A teammate asks you to act on this task: Use generated tests to explore behavior, while ensuring assertions prove requirements. Some requirements or evidence are missing. What is the strongest response?

- A. Identify the missing assumption, gather the smallest useful evidence, narrow the request, and verify affected behavior.
- B. Let the AI infer the missing requirements and make a broad change so work continues quickly.
- C. Ignore the uncertainty because the requested change appears small.

Refactoring with Copilot

Learning objectives

- Explain how Refactoring with Copilot supports a software-development task.
- Apply a bounded workflow that uses relevant context and makes assumptions visible.
- Evaluate and verify AI-supported work before relying on it.

Start with the engineering decision

Make safe, incremental refactors with local context and behavior checks. The useful outcome is a decision or change that can be checked in the context where the software actually runs. AI can help you explore options and reduce drafting effort; it cannot supply missing requirements or take responsibility for the result.

This matters in github copilot because plausible output can still conflict with project behavior, user expectations, or operational constraints. Begin with the observed problem and define what success looks like before requesting a solution. Keep assumptions visible so a teammate can challenge them.

A practical workflow

1. State the task in terms of the behavior or decision you need: make safe, incremental refactors with local context and behavior checks.
2. Supply only the relevant code, contract, evidence, and constraints; remove secrets and unrelated data.
3. Establish baseline tests; select one structural change; preserve interfaces; review all usages and run targeted checks before broader checks.
4. Ask for assumptions and alternatives when the evidence is incomplete; do not let a confident tone substitute for a source or reproduction.
5. Review the result in the project, run proportionate checks, and keep, revise, or reject it based on observable evidence.

Worked scenario

When simplifying a conditional, retain distinctions between missing, null, and empty values if callers depend on them. A useful request separates what is known from what is suspected. For example: “Here is the sanitized behavior, relevant contract, and constraint. First identify the assumptions and propose the smallest experiment that distinguishes the likely causes. Do not change unrelated interfaces.” This asks for reasoning that can be tested rather than a patch detached from evidence.

task-brief.txt

A compact brief; replace bracketed text with project-specific evidence.

Goal: Make safe, incremental refactors with local context and behavior checks.

Context: [relevant behavior, files, version, or sanitized evidence]

Constraints: [invariants, policy, compatibility, and out-of-scope work]

Request: Establish baseline tests; select one structural change; preserve interfaces; review all usages and run targeted checks before broader checks.

Check: [test, source, reproduction, or review that will verify the result]

Review and verification

- Compare the suggestion with the canonical requirement and nearby project conventions.
- Check normal, boundary, and failure behavior; inspect what the proposed change leaves untouched.
- Run the narrowest useful automated check, then broader checks when the change crosses boundaries.
- Read the complete diff and explain the result in your own words before accepting it.

A plausible answer still needs evidence

A common failure is to treat a fluent answer as proof. Refactoring with Copilot requires context and judgment: verify claims against code, tests, authoritative documentation, or a reproducible observation. If the tool lacks necessary context, ask a question or gather evidence instead of filling the gap with a guess.

Use AI to make the next engineering decision easier to inspect; keep the decision and its verification yours.

Working principle

Practice

Practice: Refactoring with Copilot Optional

Refactor a small duplicated block and document the behavior-preservation evidence.

1. Choose a small, permitted task related to Refactoring with Copilot.
2. Write the goal, relevant context, constraints, and observable acceptance criteria before consulting an AI tool.
3. Request a bounded explanation, plan, or draft; note assumptions that need confirmation.
4. Verify the result with a source, focused test, reproduction, or review checklist and record what you changed or rejected.

Constraints: Use synthetic or approved information only; never include credentials, private user data, or material you are not allowed to share.

Expected outcome: A short task brief, a reviewable AI contribution, and evidence that addresses: Refactor a small duplicated block and document the behavior-preservation evidence.

Knowledge check

Select your answers before checking them in the course reader. The answer key is intentionally not included in this ebook.

Question 1 (required). A developer is working on Refactoring with Copilot and receives a confident AI recommendation. What is the strongest next step?

- A. Apply the workflow: Establish baseline tests; select one structural change; preserve interfaces; review all usages and run targeted checks before broader checks. Then verify the outcome against project evidence.
- B. Accept the recommendation because it is detailed and sounds technically plausible.
- C. Ask the model to repeat the same answer more confidently, then proceed without a project check.

Question 2 (required). A teammate asks you to act on this task: Make safe, incremental refactors with local context and behavior checks Some requirements or evidence are missing. What is the strongest response?

- A. Identify the missing assumption, gather the smallest useful evidence, narrow the request, and verify affected behavior.
- B. Let the AI infer the missing requirements and make a broad change so work continues quickly.
- C. Ignore the uncertainty because the requested change appears small.

Copilot as a Pair Programmer

Learning objectives

- Explain how Copilot as a Pair Programmer supports a software-development task.
- Apply a bounded workflow that uses relevant context and makes assumptions visible.
- Evaluate and verify AI-supported work before relying on it.

Start with the engineering decision

Use short conversational turns to collaborate on an inspectable change. The useful outcome is a decision or change that can be checked in the context where the software actually runs. AI can help you explore options and reduce drafting effort; it cannot supply missing requirements or take responsibility for the result.

This matters in github copilot because plausible output can still conflict with project behavior, user expectations, or operational constraints. Begin with the observed problem and define what success looks like before requesting a solution. Keep assumptions visible so a teammate can challenge them.

A practical workflow

1. State the task in terms of the behavior or decision you need: use short conversational turns to collaborate on an inspectable change.
2. Supply only the relevant code, contract, evidence, and constraints; remove secrets and unrelated data.
3. Explain the goal and constraints; ask for a plan; review one step; provide correction from observed output; finish with diff and test review.
4. Ask for assumptions and alternatives when the evidence is incomplete; do not let a confident tone substitute for a source or reproduction.
5. Review the result in the project, run proportionate checks, and keep, revise, or reject it based on observable evidence.

Worked scenario

If the initial suggestion violates an existing API, point to the contract and ask for a minimal adjustment instead of accepting a second rewrite wholesale. A useful request separates what is known from what is suspected. For example: “Here is the sanitized behavior, relevant contract, and constraint. First identify the assumptions and propose the smallest experiment that distinguishes the likely causes. Do not change unrelated interfaces.” This asks for reasoning that can be tested rather than a patch detached from evidence.

task-brief.txt

A compact brief; replace bracketed text with project-specific evidence.

Goal: Use short conversational turns to collaborate on an inspectable change.

Context: [relevant behavior, files, version, or sanitized evidence]

Constraints: [invariants, policy, compatibility, and out-of-scope work]

Request: Explain the goal and constraints; ask for a plan; review one step; provide correction from observed output; finish with diff and test review.

Check: [test, source, reproduction, or review that will verify the result]

Review and verification

- Compare the suggestion with the canonical requirement and nearby project conventions.
- Check normal, boundary, and failure behavior; inspect what the proposed change leaves untouched.
- Run the narrowest useful automated check, then broader checks when the change crosses boundaries.
- Read the complete diff and explain the result in your own words before accepting it.

A plausible answer still needs evidence

A common failure is to treat a fluent answer as proof. Copilot as a Pair Programmer requires context and judgment: verify claims against code, tests, authoritative documentation, or a reproducible observation. If the tool lacks necessary context, ask a question or gather evidence instead of filling the gap with a guess.

Use AI to make the next engineering decision easier to inspect; keep the decision and its verification yours.

Working principle

Practice

Practice: Copilot as a Pair Programmer Optional

Simulate a three-turn pairing session for a bug and identify where repository evidence changes the next request.

1. Choose a small, permitted task related to Copilot as a Pair Programmer.
2. Write the goal, relevant context, constraints, and observable acceptance criteria before consulting an AI tool.
3. Request a bounded explanation, plan, or draft; note assumptions that need confirmation.
4. Verify the result with a source, focused test, reproduction, or review checklist and record what you changed or rejected.

Constraints: Use synthetic or approved information only; never include credentials, private user data, or material you are not allowed to share.

Expected outcome: A short task brief, a reviewable AI contribution, and evidence that addresses:
Simulate a three-turn pairing session for a bug and identify where repository evidence changes the next request.

Knowledge check

Select your answers before checking them in the course reader. The answer key is intentionally not included in this ebook.

Question 1 (required). A developer is working on Copilot as a Pair Programmer and receives a confident AI recommendation. What is the strongest next step?

- A. Apply the workflow: Explain the goal and constraints; ask for a plan; review one step; provide correction from observed output; finish with diff and test review. Then verify the outcome against project evidence.
- B. Accept the recommendation because it is detailed and sounds technically plausible.
- C. Ask the model to repeat the same answer more confidently, then proceed without a project check.

Question 2 (required). A teammate asks you to act on this task: Use short conversational turns to collaborate on an inspectable change. Some requirements or evidence are missing. What is the strongest response?

- A. Identify the missing assumption, gather the smallest useful evidence, narrow the request, and verify affected behavior.
- B. Let the AI infer the missing requirements and make a broad change so work continues quickly.
- C. Ignore the uncertainty because the requested change appears small.

Understanding Codex

Learning objectives

- Explain how Understanding Codex supports a software-development task.
- Apply a bounded workflow that uses relevant context and makes assumptions visible.
- Evaluate and verify AI-supported work before relying on it.

Start with the engineering decision

Understand repository agents as tools that inspect and modify files under scoped instructions. The useful outcome is a decision or change that can be checked in the context where the software actually runs. AI can help you explore options and reduce drafting effort; it cannot supply missing requirements or take responsibility for the result.

This matters in codex for software engineering because plausible output can still conflict with project behavior, user expectations, or operational constraints. Begin with the observed problem and define what success looks like before requesting a solution. Keep assumptions visible so a teammate can challenge them.

A practical workflow

1. State the task in terms of the behavior or decision you need: understand repository agents as tools that inspect and modify files under scoped instructions.
2. Supply only the relevant code, contract, evidence, and constraints; remove secrets and unrelated data.
3. Set scope and protected areas; inspect repository guidance and status; request a plan for unfamiliar work; check the resulting changes.
4. Ask for assumptions and alternatives when the evidence is incomplete; do not let a confident tone substitute for a source or reproduction.
5. Review the result in the project, run proportionate checks, and keep, revise, or reject it based on observable evidence.

Worked scenario

Before asking an agent to add a validation rule, establish current tests, related schema files, and unrelated dirty work that must be preserved. A useful request separates what is known from what is suspected. For example: “Here is the sanitized behavior, relevant contract, and constraint. First identify the assumptions and propose the smallest experiment that distinguishes the likely causes. Do not change unrelated interfaces.” This asks for reasoning that can be tested rather than a patch detached from evidence.

task-brief.txt

A compact brief; replace bracketed text with project-specific evidence.

Goal: Understand repository agents as tools that inspect and modify files under scoped instructions.

Context: [relevant behavior, files, version, or sanitized evidence]

Constraints: [invariants, policy, compatibility, and out-of-scope work]

Request: Set scope and protected areas; inspect repository guidance and status; request a plan for unfamiliar work; check the resulting changes.

Check: [test, source, reproduction, or review that will verify the result]

Review and verification

- Compare the suggestion with the canonical requirement and nearby project conventions.
- Check normal, boundary, and failure behavior; inspect what the proposed change leaves untouched.
- Run the narrowest useful automated check, then broader checks when the change crosses boundaries.
- Read the complete diff and explain the result in your own words before accepting it.

A plausible answer still needs evidence

A common failure is to treat a fluent answer as proof. Understanding Codex requires context and judgment: verify claims against code, tests, authoritative documentation, or a reproducible observation. If the tool lacks necessary context, ask a question or gather evidence instead of filling the gap with a guess.

Use AI to make the next engineering decision easier to inspect; keep the decision and its verification yours.

Working principle

Practice

Practice: Understanding Codex Optional

Write a repository task brief with scope, exclusions, validation commands, and review criteria.

1. Choose a small, permitted task related to Understanding Codex.
2. Write the goal, relevant context, constraints, and observable acceptance criteria before consulting an AI tool.
3. Request a bounded explanation, plan, or draft; note assumptions that need confirmation.
4. Verify the result with a source, focused test, reproduction, or review checklist and record what you changed or rejected.

Constraints: Use synthetic or approved information only; never include credentials, private user data, or material you are not allowed to share.

Expected outcome: A short task brief, a reviewable AI contribution, and evidence that addresses: Write a repository task brief with scope, exclusions, validation commands, and review criteria.

Knowledge check

Select your answers before checking them in the course reader. The answer key is intentionally not included in this ebook.

Question 1 (required). A developer is working on Understanding Codex and receives a confident AI recommendation. What is the strongest next step?

- A. Apply the workflow: Set scope and protected areas; inspect repository guidance and status; request a plan for unfamiliar work; check the resulting changes. Then verify the outcome against project evidence.
- B. Accept the recommendation because it is detailed and sounds technically plausible.
- C. Ask the model to repeat the same answer more confidently, then proceed without a project check.

Question 2 (required). A teammate asks you to act on this task: Understand repository agents as tools that inspect and modify files under scoped instructions. Some requirements or evidence are missing. What is the strongest response?

- A. Identify the missing assumption, gather the smallest useful evidence, narrow the request, and verify affected behavior.
- B. Let the AI infer the missing requirements and make a broad change so work continues quickly.
- C. Ignore the uncertainty because the requested change appears small.

Generating and Editing Code

Learning objectives

- Explain how Generating and Editing Code supports a software-development task.
- Apply a bounded workflow that uses relevant context and makes assumptions visible.
- Evaluate and verify AI-supported work before relying on it.

Start with the engineering decision

Delegate a narrow code change with explicit behavior and a reviewable finish. The useful outcome is a decision or change that can be checked in the context where the software actually runs. AI can help you explore options and reduce drafting effort; it cannot supply missing requirements or take responsibility for the result.

This matters in codex for software engineering because plausible output can still conflict with project behavior, user expectations, or operational constraints. Begin with the observed problem and define what success looks like before requesting a solution. Keep assumptions visible so a teammate can challenge them.

A practical workflow

1. State the task in terms of the behavior or decision you need: delegate a narrow code change with explicit behavior and a reviewable finish.
2. Supply only the relevant code, contract, evidence, and constraints; remove secrets and unrelated data.
3. Describe current and desired behavior; identify relevant files and constraints; ask for a focused change; inspect the diff and run relevant checks.
4. Ask for assumptions and alternatives when the evidence is incomplete; do not let a confident tone substitute for a source or reproduction.
5. Review the result in the project, run proportionate checks, and keep, revise, or reject it based on observable evidence.

Worked scenario

A request to add a loading state should specify existing component patterns, accessible status text, and behavior on failure. A useful request separates what is known from what is suspected. For example: "Here is the sanitized behavior, relevant contract, and constraint. First identify the assumptions and propose the smallest experiment that distinguishes the likely causes. Do not change unrelated interfaces." This asks for reasoning that can be tested rather than a patch detached from evidence.

task-brief.txt

A compact brief; replace bracketed text with project-specific evidence.

Goal: Delegate a narrow code change with explicit behavior and a reviewable finish.

Context: [relevant behavior, files, version, or sanitized evidence]

Constraints: [invariants, policy, compatibility, and out-of-scope work]

Request: Describe current and desired behavior; identify relevant files and constraints; ask for a focused change; inspect the diff and run relevant checks.

Check: [test, source, reproduction, or review that will verify the result]

Review and verification

- Compare the suggestion with the canonical requirement and nearby project conventions.
- Check normal, boundary, and failure behavior; inspect what the proposed change leaves untouched.
- Run the narrowest useful automated check, then broader checks when the change crosses boundaries.
- Read the complete diff and explain the result in your own words before accepting it.

A plausible answer still needs evidence

A common failure is to treat a fluent answer as proof. Generating and Editing Code requires context and judgment: verify claims against code, tests, authoritative documentation, or a reproducible observation. If the tool lacks necessary context, ask a question or gather evidence instead of filling the gap with a guess.

Use AI to make the next engineering decision easier to inspect; keep the decision and its verification yours.

Working principle

Practice

Practice: Generating and Editing Code Optional

Define a bounded code edit and acceptance checklist before asking an agent to implement it.

1. Choose a small, permitted task related to Generating and Editing Code.
2. Write the goal, relevant context, constraints, and observable acceptance criteria before consulting an AI tool.
3. Request a bounded explanation, plan, or draft; note assumptions that need confirmation.
4. Verify the result with a source, focused test, reproduction, or review checklist and record what you changed or rejected.

Constraints: Use synthetic or approved information only; never include credentials, private user data, or material you are not allowed to share.

Expected outcome: A short task brief, a reviewable AI contribution, and evidence that addresses: Define a bounded code edit and acceptance checklist before asking an agent to implement it.

Knowledge check

Select your answers before checking them in the course reader. The answer key is intentionally not included in this ebook.

Question 1 (required). A developer is working on Generating and Editing Code and receives a confident AI recommendation. What is the strongest next step?

- A. Apply the workflow: Describe current and desired behavior; identify relevant files and constraints; ask for a focused change; inspect the diff and run relevant checks. Then verify the outcome against project evidence.
- B. Accept the recommendation because it is detailed and sounds technically plausible.
- C. Ask the model to repeat the same answer more confidently, then proceed without a project check.

Question 2 (required). A teammate asks you to act on this task: Delegate a narrow code change with explicit behavior and a reviewable finish Some requirements or evidence are missing. What is the strongest response?

- A. Identify the missing assumption, gather the smallest useful evidence, narrow the request, and verify affected behavior.
- B. Let the AI infer the missing requirements and make a broad change so work continues quickly.
- C. Ignore the uncertainty because the requested change appears small.

Multi-File Changes

Learning objectives

- Explain how Multi-File Changes supports a software-development task.
- Apply a bounded workflow that uses relevant context and makes assumptions visible.
- Evaluate and verify AI-supported work before relying on it.

Start with the engineering decision

Coordinate related changes across layers without losing contract consistency. The useful outcome is a decision or change that can be checked in the context where the software actually runs. AI can help you explore options and reduce drafting effort; it cannot supply missing requirements or take responsibility for the result.

This matters in codex for software engineering because plausible output can still conflict with project behavior, user expectations, or operational constraints. Begin with the observed problem and define what success looks like before requesting a solution. Keep assumptions visible so a teammate can challenge them.

A practical workflow

1. State the task in terms of the behavior or decision you need: coordinate related changes across layers without losing contract consistency.
2. Supply only the relevant code, contract, evidence, and constraints; remove secrets and unrelated data.
3. Map the contract across caller, service, API, and tests; plan sequence; keep names and types aligned; verify integration paths.
4. Ask for assumptions and alternatives when the evidence is incomplete; do not let a confident tone substitute for a source or reproduction.
5. Review the result in the project, run proportionate checks, and keep, revise, or reject it based on observable evidence.

Worked scenario

Adding an optional profile field may touch validation, persistence, response shaping, and UI; trace each boundary rather than editing only the form. A useful request separates what is known from what is suspected. For example: “Here is the sanitized behavior, relevant contract, and constraint. First identify the assumptions and propose the smallest experiment that distinguishes the likely causes. Do not change unrelated interfaces.” This asks for reasoning that can be tested rather than a patch detached from evidence.

task-brief.txt

A compact brief; replace bracketed text with project-specific evidence.

Goal: Coordinate related changes across layers without losing contract consistency.

Context: [relevant behavior, files, version, or sanitized evidence]

Constraints: [invariants, policy, compatibility, and out-of-scope work]

Request: Map the contract across caller, service, API, and tests; plan sequence; keep names and types aligned; verify integration paths.

Check: [test, source, reproduction, or review that will verify the result]

Review and verification

- Compare the suggestion with the canonical requirement and nearby project conventions.
- Check normal, boundary, and failure behavior; inspect what the proposed change leaves untouched.
- Run the narrowest useful automated check, then broader checks when the change crosses boundaries.
- Read the complete diff and explain the result in your own words before accepting it.

A plausible answer still needs evidence

A common failure is to treat a fluent answer as proof. Multi-File Changes requires context and judgment: verify claims against code, tests, authoritative documentation, or a reproducible observation. If the tool lacks necessary context, ask a question or gather evidence instead of filling the gap with a guess.

Use AI to make the next engineering decision easier to inspect; keep the decision and its verification yours.

Working principle

Practice

Practice: Multi-File Changes Optional

Draw the files and data contract involved in a small multi-layer change; name a check at each boundary.

1. Choose a small, permitted task related to Multi-File Changes.
2. Write the goal, relevant context, constraints, and observable acceptance criteria before consulting an AI tool.
3. Request a bounded explanation, plan, or draft; note assumptions that need confirmation.
4. Verify the result with a source, focused test, reproduction, or review checklist and record what you changed or rejected.

Constraints: Use synthetic or approved information only; never include credentials, private user data, or material you are not allowed to share.

Expected outcome: A short task brief, a reviewable AI contribution, and evidence that addresses: Draw the files and data contract involved in a small multi-layer change; name a check at each boundary.

Knowledge check

Select your answers before checking them in the course reader. The answer key is intentionally not included in this ebook.

Question 1 (required). A developer is working on Multi-File Changes and receives a confident AI recommendation. What is the strongest next step?

- A. Apply the workflow: Map the contract across caller, service, API, and tests; plan sequence; keep names and types aligned; verify integration paths. Then verify the outcome against project evidence.
- B. Accept the recommendation because it is detailed and sounds technically plausible.
- C. Ask the model to repeat the same answer more confidently, then proceed without a project check.

Question 2 (required). A teammate asks you to act on this task: Coordinate related changes across layers without losing contract consistency Some requirements or evidence are missing. What is the strongest response?

- A. Identify the missing assumption, gather the smallest useful evidence, narrow the request, and verify affected behavior.
- B. Let the AI infer the missing requirements and make a broad change so work continues quickly.
- C. Ignore the uncertainty because the requested change appears small.

Explaining Existing Codebases

Learning objectives

- Explain how Explaining Existing Codebases supports a software-development task.
- Apply a bounded workflow that uses relevant context and makes assumptions visible.
- Evaluate and verify AI-supported work before relying on it.

Start with the engineering decision

Build an evidence-backed map of an unfamiliar repository. The useful outcome is a decision or change that can be checked in the context where the software actually runs. AI can help you explore options and reduce drafting effort; it cannot supply missing requirements or take responsibility for the result.

This matters in codex for software engineering because plausible output can still conflict with project behavior, user expectations, or operational constraints. Begin with the observed problem and define what success looks like before requesting a solution. Keep assumptions visible so a teammate can challenge them.

A practical workflow

1. State the task in terms of the behavior or decision you need: build an evidence-backed map of an unfamiliar repository.
2. Supply only the relevant code, contract, evidence, and constraints; remove secrets and unrelated data.
3. Start at entry points and package scripts; follow a real request or state transition; confirm summaries with definitions and call sites.
4. Ask for assumptions and alternatives when the evidence is incomplete; do not let a confident tone substitute for a source or reproduction.
5. Review the result in the project, run proportionate checks, and keep, revise, or reject it based on observable evidence.

Worked scenario

Trace a login request from route through validation and persistence to the client response, noting where authorization is enforced. A useful request separates what is known from what is suspected. For example: “Here is the sanitized behavior, relevant contract, and constraint. First identify the assumptions and propose the smallest experiment that distinguishes the likely causes. Do not change unrelated interfaces.” This asks for reasoning that can be tested rather than a patch detached from evidence.

task-brief.txt

A compact brief; replace bracketed text with project-specific evidence.

Goal: Build an evidence-backed map of an unfamiliar repository.

Context: [relevant behavior, files, version, or sanitized evidence]

Constraints: [invariants, policy, compatibility, and out-of-scope work]

Request: Start at entry points and package scripts; follow a real request or state transition; confirm summaries with definitions and call sites.

Check: [test, source, reproduction, or review that will verify the result]

Review and verification

- Compare the suggestion with the canonical requirement and nearby project conventions.
- Check normal, boundary, and failure behavior; inspect what the proposed change leaves untouched.
- Run the narrowest useful automated check, then broader checks when the change crosses boundaries.
- Read the complete diff and explain the result in your own words before accepting it.

A plausible answer still needs evidence

A common failure is to treat a fluent answer as proof. Explaining Existing Codebases requires context and judgment: verify claims against code, tests, authoritative documentation, or a reproducible observation. If the tool lacks necessary context, ask a question or gather evidence instead of filling the gap with a guess.

Use AI to make the next engineering decision easier to inspect; keep the decision and its verification yours.

Working principle

Practice

Practice: Explaining Existing Codebases Optional

Ask for a codebase explanation, then verify three claims against named files or symbols.

1. Choose a small, permitted task related to Explaining Existing Codebases.
2. Write the goal, relevant context, constraints, and observable acceptance criteria before consulting an AI tool.
3. Request a bounded explanation, plan, or draft; note assumptions that need confirmation.
4. Verify the result with a source, focused test, reproduction, or review checklist and record what you changed or rejected.

Constraints: Use synthetic or approved information only; never include credentials, private user data, or material you are not allowed to share.

Expected outcome: A short task brief, a reviewable AI contribution, and evidence that addresses: Ask for a codebase explanation, then verify three claims against named files or symbols.

Knowledge check

Select your answers before checking them in the course reader. The answer key is intentionally not included in this ebook.

Question 1 (required). A developer is working on Explaining Existing Codebases and receives a confident AI recommendation. What is the strongest next step?

- A. Apply the workflow: Start at entry points and package scripts; follow a real request or state transition; confirm summaries with definitions and call sites. Then verify the outcome against project evidence.
- B. Accept the recommendation because it is detailed and sounds technically plausible.
- C. Ask the model to repeat the same answer more confidently, then proceed without a project check.

Question 2 (required). A teammate asks you to act on this task: Build an evidence-backed map of an unfamiliar repository. Some requirements or evidence are missing. What is the strongest response?

- A. Identify the missing assumption, gather the smallest useful evidence, narrow the request, and verify affected behavior.
- B. Let the AI infer the missing requirements and make a broad change so work continues quickly.
- C. Ignore the uncertainty because the requested change appears small.

Handling Larger Development Tasks

Learning objectives

- Explain how Handling Larger Development Tasks supports a software-development task.
- Apply a bounded workflow that uses relevant context and makes assumptions visible.
- Evaluate and verify AI-supported work before relying on it.

Start with the engineering decision

Reduce a large task into safe increments with clear stop and review points. The useful outcome is a decision or change that can be checked in the context where the software actually runs. AI can help you explore options and reduce drafting effort; it cannot supply missing requirements or take responsibility for the result.

This matters in codex for software engineering because plausible output can still conflict with project behavior, user expectations, or operational constraints. Begin with the observed problem and define what success looks like before requesting a solution. Keep assumptions visible so a teammate can challenge them.

A practical workflow

1. State the task in terms of the behavior or decision you need: reduce a large task into safe increments with clear stop and review points.
2. Supply only the relevant code, contract, evidence, and constraints; remove secrets and unrelated data.
3. Inventory requirements and existing behavior; split into independently verifiable steps; implement one slice at a time; test and reassess.
4. Ask for assumptions and alternatives when the evidence is incomplete; do not let a confident tone substitute for a source or reproduction.
5. Review the result in the project, run proportionate checks, and keep, revise, or reject it based on observable evidence.

Worked scenario

For a reporting feature, first define data shape and access control, then add query logic, API response, and UI as separate reviewed steps. A useful request separates what is known from what is suspected. For example: “Here is the sanitized behavior, relevant contract, and constraint. First identify the assumptions and propose the smallest experiment that distinguishes the likely causes. Do not change unrelated interfaces.” This asks for reasoning that can be tested rather than a patch detached from evidence.

task-brief.txt

A compact brief; replace bracketed text with project-specific evidence.

Goal: Reduce a large task into safe increments with clear stop and review points.

Context: [relevant behavior, files, version, or sanitized evidence]

Constraints: [invariants, policy, compatibility, and out-of-scope work]

Request: Inventory requirements and existing behavior; split into independently verifiable steps; implement one slice at a time; test and reassess.

Check: [test, source, reproduction, or review that will verify the result]

Review and verification

- Compare the suggestion with the canonical requirement and nearby project conventions.
- Check normal, boundary, and failure behavior; inspect what the proposed change leaves untouched.
- Run the narrowest useful automated check, then broader checks when the change crosses boundaries.
- Read the complete diff and explain the result in your own words before accepting it.

A plausible answer still needs evidence

A common failure is to treat a fluent answer as proof. Handling Larger Development Tasks requires context and judgment: verify claims against code, tests, authoritative documentation, or a reproducible observation. If the tool lacks necessary context, ask a question or gather evidence instead of filling the gap with a guess.

Use AI to make the next engineering decision easier to inspect; keep the decision and its verification yours.

Working principle

Practice

Practice: Handling Larger Development Tasks Optional

Decompose a larger feature into increments with dependencies, acceptance checks, and explicit out-of-scope areas.

1. Choose a small, permitted task related to Handling Larger Development Tasks.
2. Write the goal, relevant context, constraints, and observable acceptance criteria before consulting an AI tool.
3. Request a bounded explanation, plan, or draft; note assumptions that need confirmation.
4. Verify the result with a source, focused test, reproduction, or review checklist and record what you changed or rejected.

Constraints: Use synthetic or approved information only; never include credentials, private user data, or material you are not allowed to share.

Expected outcome: A short task brief, a reviewable AI contribution, and evidence that addresses: Decompose a larger feature into increments with dependencies, acceptance checks, and explicit out-of-scope areas.

Knowledge check

Select your answers before checking them in the course reader. The answer key is intentionally not included in this ebook.

Question 1 (required). A developer is working on Handling Larger Development Tasks and receives a confident AI recommendation. What is the strongest next step?

- A. Apply the workflow: Inventory requirements and existing behavior; split into independently verifiable steps; implement one slice at a time; test and reassess. Then verify the outcome against project evidence.
- B. Accept the recommendation because it is detailed and sounds technically plausible.
- C. Ask the model to repeat the same answer more confidently, then proceed without a project check.

Question 2 (required). A teammate asks you to act on this task: Reduce a large task into safe increments with clear stop and review points. Some requirements or evidence are missing. What is the strongest response?

- A. Identify the missing assumption, gather the smallest useful evidence, narrow the request, and verify affected behavior.
- B. Let the AI infer the missing requirements and make a broad change so work continues quickly.
- C. Ignore the uncertainty because the requested change appears small.

ChatGPT vs GitHub Copilot vs Codex

Learning objectives

- Explain how ChatGPT vs GitHub Copilot vs Codex supports a software-development task.
- Apply a bounded workflow that uses relevant context and makes assumptions visible.
- Evaluate and verify AI-supported work before relying on it.

Start with the engineering decision

Choose among conversation, in-editor assistance, and repository-level work by task shape. The useful outcome is a decision or change that can be checked in the context where the software actually runs. AI can help you explore options and reduce drafting effort; it cannot supply missing requirements or take responsibility for the result.

This matters in codex for software engineering because plausible output can still conflict with project behavior, user expectations, or operational constraints. Begin with the observed problem and define what success looks like before requesting a solution. Keep assumptions visible so a teammate can challenge them.

A practical workflow

1. State the task in terms of the behavior or decision you need: choose among conversation, in-editor assistance, and repository-level work by task shape.
2. Supply only the relevant code, contract, evidence, and constraints; remove secrets and unrelated data.
3. Compare context access, change size, feedback cycle, and risk; choose the smallest capable workflow; verify in the repository.
4. Ask for assumptions and alternatives when the evidence is incomplete; do not let a confident tone substitute for a source or reproduction.
5. Review the result in the project, run proportionate checks, and keep, revise, or reject it based on observable evidence.

Worked scenario

Use conversation to explore a design, inline completion for a local expression, and a repository agent for a bounded cross-file task with tests. A useful request separates what is known from what is suspected. For example: “Here is the sanitized behavior, relevant contract, and constraint. First identify the assumptions and propose the smallest experiment that distinguishes the likely causes. Do not change unrelated interfaces.” This asks for reasoning that can be tested rather than a patch detached from evidence.

task-brief.txt

A compact brief; replace bracketed text with project-specific evidence.

Goal: Choose among conversation, in-editor assistance, and repository-level work by task shape.

Context: [relevant behavior, files, version, or sanitized evidence]

Constraints: [invariants, policy, compatibility, and out-of-scope work]

Request: Compare context access, change size, feedback cycle, and risk; choose the smallest capable workflow; verify in the repository.

Check: [test, source, reproduction, or review that will verify the result]

Review and verification

- Compare the suggestion with the canonical requirement and nearby project conventions.
- Check normal, boundary, and failure behavior; inspect what the proposed change leaves untouched.
- Run the narrowest useful automated check, then broader checks when the change crosses boundaries.
- Read the complete diff and explain the result in your own words before accepting it.

A plausible answer still needs evidence

A common failure is to treat a fluent answer as proof. ChatGPT vs GitHub Copilot vs Codex requires context and judgment: verify claims against code, tests, authoritative documentation, or a reproducible observation. If the tool lacks necessary context, ask a question or gather evidence instead of filling the gap with a guess.

Use AI to make the next engineering decision easier to inspect; keep the decision and its verification yours.

Working principle

Practice

Practice: ChatGPT vs GitHub Copilot vs Codex Optional

Map four tasks to a tool workflow and justify each choice by context and verification needs.

1. Choose a small, permitted task related to ChatGPT vs GitHub Copilot vs Codex.
2. Write the goal, relevant context, constraints, and observable acceptance criteria before consulting an AI tool.
3. Request a bounded explanation, plan, or draft; note assumptions that need confirmation.
4. Verify the result with a source, focused test, reproduction, or review checklist and record what you changed or rejected.

Constraints: Use synthetic or approved information only; never include credentials, private user data, or material you are not allowed to share.

Expected outcome: A short task brief, a reviewable AI contribution, and evidence that addresses: Map four tasks to a tool workflow and justify each choice by context and verification needs.

Knowledge check

Select your answers before checking them in the course reader. The answer key is intentionally not included in this ebook.

Question 1 (required). A developer is working on ChatGPT vs GitHub Copilot vs Codex and receives a confident AI recommendation. What is the strongest next step?

- A. Apply the workflow: Compare context access, change size, feedback cycle, and risk; choose the smallest capable workflow; verify in the repository. Then verify the outcome against project evidence.
- B. Accept the recommendation because it is detailed and sounds technically plausible.
- C. Ask the model to repeat the same answer more confidently, then proceed without a project check.

Question 2 (required). A teammate asks you to act on this task: Choose among conversation, in-editor assistance, and repository-level work by task shape. Some requirements or evidence are missing. What is the strongest response?

- A. Identify the missing assumption, gather the smallest useful evidence, narrow the request, and verify affected behavior.
- B. Let the AI infer the missing requirements and make a broad change so work continues quickly.
- C. Ignore the uncertainty because the requested change appears small.

Planning a Software Project with ChatGPT

Learning objectives

- Explain how Planning a Software Project with ChatGPT supports a software-development task.
- Apply a bounded workflow that uses relevant context and makes assumptions visible.
- Evaluate and verify AI-supported work before relying on it.

Start with the engineering decision

Use AI to interrogate a project idea and create a validated planning baseline. The useful outcome is a decision or change that can be checked in the context where the software actually runs. AI can help you explore options and reduce drafting effort; it cannot supply missing requirements or take responsibility for the result.

This matters in planning the project with ai because plausible output can still conflict with project behavior, user expectations, or operational constraints. Begin with the observed problem and define what success looks like before requesting a solution. Keep assumptions visible so a teammate can challenge them.

A practical workflow

1. State the task in terms of the behavior or decision you need: use AI to interrogate a project idea and create a validated planning baseline.
2. Supply only the relevant code, contract, evidence, and constraints; remove secrets and unrelated data.
3. Describe users and outcomes; separate known facts from assumptions; ask for questions and options; confirm scope with stakeholders.
4. Ask for assumptions and alternatives when the evidence is incomplete; do not let a confident tone substitute for a source or reproduction.
5. Review the result in the project, run proportionate checks, and keep, revise, or reject it based on observable evidence.

Worked scenario

For a booking app, clarify cancellation rules, time zones, and ownership before asking for entities or endpoints. A useful request separates what is known from what is suspected. For example: “Here is the sanitized behavior, relevant contract, and constraint. First identify the assumptions and propose the smallest experiment that distinguishes the likely causes. Do not change unrelated interfaces.” This asks for reasoning that can be tested rather than a patch detached from evidence.

task-brief.txt

A compact brief; replace bracketed text with project-specific evidence.

Goal: Use AI to interrogate a project idea and create a validated planning baseline.

Context: [relevant behavior, files, version, or sanitized evidence]

Constraints: [invariants, policy, compatibility, and out-of-scope work]

Request: Describe users and outcomes; separate known facts from assumptions; ask for questions and options; confirm scope with stakeholders.

Check: [test, source, reproduction, or review that will verify the result]

Review and verification

- Compare the suggestion with the canonical requirement and nearby project conventions.
- Check normal, boundary, and failure behavior; inspect what the proposed change leaves untouched.
- Run the narrowest useful automated check, then broader checks when the change crosses boundaries.
- Read the complete diff and explain the result in your own words before accepting it.

A plausible answer still needs evidence

A common failure is to treat a fluent answer as proof. Planning a Software Project with ChatGPT requires context and judgment: verify claims against code, tests, authoritative documentation, or a reproducible observation. If the tool lacks necessary context, ask a question or gather evidence instead of filling the gap with a guess.

Use AI to make the next engineering decision easier to inspect; keep the decision and its verification yours.

Working principle

Practice

Practice: Planning a Software Project with ChatGPT Optional

Turn a project idea into goals, open questions, assumptions, and a first validation activity.

1. Choose a small, permitted task related to Planning a Software Project with ChatGPT.
2. Write the goal, relevant context, constraints, and observable acceptance criteria before consulting an AI tool.
3. Request a bounded explanation, plan, or draft; note assumptions that need confirmation.
4. Verify the result with a source, focused test, reproduction, or review checklist and record what you changed or rejected.

Constraints: Use synthetic or approved information only; never include credentials, private user data, or material you are not allowed to share.

Expected outcome: A short task brief, a reviewable AI contribution, and evidence that addresses: Turn a project idea into goals, open questions, assumptions, and a first validation activity.

Knowledge check

Select your answers before checking them in the course reader. The answer key is intentionally not included in this ebook.

Question 1 (required). A developer is working on Planning a Software Project with ChatGPT and receives a confident AI recommendation. What is the strongest next step?

- A. Apply the workflow: Describe users and outcomes; separate known facts from assumptions; ask for questions and options; confirm scope with stakeholders. Then verify the outcome against project evidence.
- B. Accept the recommendation because it is detailed and sounds technically plausible.
- C. Ask the model to repeat the same answer more confidently, then proceed without a project check.

Question 2 (required). A teammate asks you to act on this task: Use AI to interrogate a project idea and create a validated planning baseline. Some requirements or evidence are missing. What is the strongest response?

- A. Identify the missing assumption, gather the smallest useful evidence, narrow the request, and verify affected behavior.
- B. Let the AI infer the missing requirements and make a broad change so work continues quickly.
- C. Ignore the uncertainty because the requested change appears small.

Turning Requirements into Tasks

Learning objectives

- Explain how Turning Requirements into Tasks supports a software-development task.
- Apply a bounded workflow that uses relevant context and makes assumptions visible.
- Evaluate and verify AI-supported work before relying on it.

Start with the engineering decision

Translate requirements into small tasks with observable acceptance criteria. The useful outcome is a decision or change that can be checked in the context where the software actually runs. AI can help you explore options and reduce drafting effort; it cannot supply missing requirements or take responsibility for the result.

This matters in planning the project with ai because plausible output can still conflict with project behavior, user expectations, or operational constraints. Begin with the observed problem and define what success looks like before requesting a solution. Keep assumptions visible so a teammate can challenge them.

A practical workflow

1. State the task in terms of the behavior or decision you need: translate requirements into small tasks with observable acceptance criteria.
2. Supply only the relevant code, contract, evidence, and constraints; remove secrets and unrelated data.
3. Extract actors, triggers, outcomes, errors, and constraints; split by vertical behavior; make each task testable and dependency-aware.
4. Ask for assumptions and alternatives when the evidence is incomplete; do not let a confident tone substitute for a source or reproduction.
5. Review the result in the project, run proportionate checks, and keep, revise, or reject it based on observable evidence.

Worked scenario

A requirement to reset a password becomes request, token validation, expiry handling, notification, and audit considerations-not just a button. A useful request separates what is known from what is suspected. For example: “Here is the sanitized behavior, relevant contract, and constraint. First identify the assumptions and propose the smallest experiment that distinguishes the likely causes. Do not change unrelated interfaces.” This asks for reasoning that can be tested rather than a patch detached from evidence.

task-brief.txt

A compact brief; replace bracketed text with project-specific evidence.

Goal: Translate requirements into small tasks with observable acceptance criteria.

Context: [relevant behavior, files, version, or sanitized evidence]

Constraints: [invariants, policy, compatibility, and out-of-scope work]

Request: Extract actors, triggers, outcomes, errors, and constraints; split by vertical behavior; make each task testable and dependency-aware.

Check: [test, source, reproduction, or review that will verify the result]

Review and verification

- Compare the suggestion with the canonical requirement and nearby project conventions.
- Check normal, boundary, and failure behavior; inspect what the proposed change leaves untouched.
- Run the narrowest useful automated check, then broader checks when the change crosses boundaries.
- Read the complete diff and explain the result in your own words before accepting it.

A plausible answer still needs evidence

A common failure is to treat a fluent answer as proof. Turning Requirements into Tasks requires context and judgment: verify claims against code, tests, authoritative documentation, or a reproducible observation. If the tool lacks necessary context, ask a question or gather evidence instead of filling the gap with a guess.

Use AI to make the next engineering decision easier to inspect; keep the decision and its verification yours.

Working principle

Practice

Practice: Turning Requirements into Tasks Optional

Break one requirement into tasks; ensure each has a verifiable outcome and negative case.

1. Choose a small, permitted task related to Turning Requirements into Tasks.
2. Write the goal, relevant context, constraints, and observable acceptance criteria before consulting an AI tool.
3. Request a bounded explanation, plan, or draft; note assumptions that need confirmation.
4. Verify the result with a source, focused test, reproduction, or review checklist and record what you changed or rejected.

Constraints: Use synthetic or approved information only; never include credentials, private user data, or material you are not allowed to share.

Expected outcome: A short task brief, a reviewable AI contribution, and evidence that addresses: Break one requirement into tasks; ensure each has a verifiable outcome and negative case.

Knowledge check

Select your answers before checking them in the course reader. The answer key is intentionally not included in this ebook.

Question 1 (required). A developer is working on Turning Requirements into Tasks and receives a confident AI recommendation. What is the strongest next step?

- A. Apply the workflow: Extract actors, triggers, outcomes, errors, and constraints; split by vertical behavior; make each task testable and dependency-aware. Then verify the outcome against project evidence.
- B. Accept the recommendation because it is detailed and sounds technically plausible.
- C. Ask the model to repeat the same answer more confidently, then proceed without a project check.

Question 2 (required). A teammate asks you to act on this task: Translate requirements into small tasks with observable acceptance criteria. Some requirements or evidence are missing. What is the strongest response?

- A. Identify the missing assumption, gather the smallest useful evidence, narrow the request, and verify affected behavior.
- B. Let the AI infer the missing requirements and make a broad change so work continues quickly.
- C. Ignore the uncertainty because the requested change appears small.

Designing Features and User Flows

Learning objectives

- Explain how Designing Features and User Flows supports a software-development task.
- Apply a bounded workflow that uses relevant context and makes assumptions visible.
- Evaluate and verify AI-supported work before relying on it.

Start with the engineering decision

Model user journeys, alternate paths, and failure states before implementation. The useful outcome is a decision or change that can be checked in the context where the software actually runs. AI can help you explore options and reduce drafting effort; it cannot supply missing requirements or take responsibility for the result.

This matters in planning the project with ai because plausible output can still conflict with project behavior, user expectations, or operational constraints. Begin with the observed problem and define what success looks like before requesting a solution. Keep assumptions visible so a teammate can challenge them.

A practical workflow

1. State the task in terms of the behavior or decision you need: model user journeys, alternate paths, and failure states before implementation.
2. Supply only the relevant code, contract, evidence, and constraints; remove secrets and unrelated data.
3. Name user and goal; map entry, decisions, success, cancellation, and errors; validate flows with product rules and accessibility needs.
4. Ask for assumptions and alternatives when the evidence is incomplete; do not let a confident tone substitute for a source or reproduction.
5. Review the result in the project, run proportionate checks, and keep, revise, or reject it based on observable evidence.

Worked scenario

An upload flow includes file selection, progress, rejection for size/type, retry, and confirmation rather than only the happy path. A useful request separates what is known from what is suspected. For example: “Here is the sanitized behavior, relevant contract, and constraint. First identify the assumptions and propose the smallest experiment that distinguishes the likely causes. Do not change unrelated interfaces.” This asks for reasoning that can be tested rather than a patch detached from evidence.

task-brief.txt

A compact brief; replace bracketed text with project-specific evidence.

Goal: Model user journeys, alternate paths, and failure states before implementation.

Context: [relevant behavior, files, version, or sanitized evidence]

Constraints: [invariants, policy, compatibility, and out-of-scope work]

Request: Name user and goal; map entry, decisions, success, cancellation, and errors; validate flows with product rules and accessibility needs.

Check: [test, source, reproduction, or review that will verify the result]

Review and verification

- Compare the suggestion with the canonical requirement and nearby project conventions.
- Check normal, boundary, and failure behavior; inspect what the proposed change leaves untouched.
- Run the narrowest useful automated check, then broader checks when the change crosses boundaries.
- Read the complete diff and explain the result in your own words before accepting it.

A plausible answer still needs evidence

A common failure is to treat a fluent answer as proof. Designing Features and User Flows requires context and judgment: verify claims against code, tests, authoritative documentation, or a reproducible observation. If the tool lacks necessary context, ask a question or gather evidence instead of filling the gap with a guess.

Use AI to make the next engineering decision easier to inspect; keep the decision and its verification yours.

Working principle

Practice

Practice: Designing Features and User Flows Optional

Create a user flow with at least one error and one recovery path and identify unresolved policy choices.

1. Choose a small, permitted task related to Designing Features and User Flows.
2. Write the goal, relevant context, constraints, and observable acceptance criteria before consulting an AI tool.
3. Request a bounded explanation, plan, or draft; note assumptions that need confirmation.
4. Verify the result with a source, focused test, reproduction, or review checklist and record what you changed or rejected.

Constraints: Use synthetic or approved information only; never include credentials, private user data, or material you are not allowed to share.

Expected outcome: A short task brief, a reviewable AI contribution, and evidence that addresses: Create a user flow with at least one error and one recovery path and identify unresolved policy choices.

Knowledge check

Select your answers before checking them in the course reader. The answer key is intentionally not included in this ebook.

Question 1 (required). A developer is working on Designing Features and User Flows and receives a confident AI recommendation. What is the strongest next step?

- A. Apply the workflow: Name user and goal; map entry, decisions, success, cancellation, and errors; validate flows with product rules and accessibility needs. Then verify the outcome against project evidence.
- B. Accept the recommendation because it is detailed and sounds technically plausible.
- C. Ask the model to repeat the same answer more confidently, then proceed without a project check.

Question 2 (required). A teammate asks you to act on this task: Model user journeys, alternate paths, and failure states before implementation. Some requirements or evidence are missing. What is the strongest response?

- A. Identify the missing assumption, gather the smallest useful evidence, narrow the request, and verify affected behavior.
- B. Let the AI infer the missing requirements and make a broad change so work continues quickly.
- C. Ignore the uncertainty because the requested change appears small.

Choosing Technologies and Architecture

Learning objectives

- Explain how Choosing Technologies and Architecture supports a software-development task.
- Apply a bounded workflow that uses relevant context and makes assumptions visible.
- Evaluate and verify AI-supported work before relying on it.

Start with the engineering decision

Evaluate technology choices against project constraints and team capability. The useful outcome is a decision or change that can be checked in the context where the software actually runs. AI can help you explore options and reduce drafting effort; it cannot supply missing requirements or take responsibility for the result.

This matters in planning the project with ai because plausible output can still conflict with project behavior, user expectations, or operational constraints. Begin with the observed problem and define what success looks like before requesting a solution. Keep assumptions visible so a teammate can challenge them.

A practical workflow

1. State the task in terms of the behavior or decision you need: evaluate technology choices against project constraints and team capability.
2. Supply only the relevant code, contract, evidence, and constraints; remove secrets and unrelated data.
3. List requirements and nonfunctional needs; compare viable options using consistent criteria; check maintenance, deployment, and migration costs.
4. Ask for assumptions and alternatives when the evidence is incomplete; do not let a confident tone substitute for a source or reproduction.
5. Review the result in the project, run proportionate checks, and keep, revise, or reject it based on observable evidence.

Worked scenario

A small internal dashboard may favor the team's familiar stack over a new service if new operational cost has no measured benefit. A useful request separates what is known from what is suspected. For example: "Here is the sanitized behavior, relevant contract, and constraint. First identify the assumptions and propose the smallest experiment that distinguishes the likely causes. Do not change unrelated interfaces." This asks for reasoning that can be tested rather than a patch detached from evidence.

task-brief.txt

A compact brief; replace bracketed text with project-specific evidence.

Goal: Evaluate technology choices against project constraints and team capability.

Context: [relevant behavior, files, version, or sanitized evidence]

Constraints: [invariants, policy, compatibility, and out-of-scope work]

Request: List requirements and nonfunctional needs; compare viable options using consistent criteria; check maintenance, deployment, and migration costs.

Check: [test, source, reproduction, or review that will verify the result]

Review and verification

- Compare the suggestion with the canonical requirement and nearby project conventions.
- Check normal, boundary, and failure behavior; inspect what the proposed change leaves untouched.
- Run the narrowest useful automated check, then broader checks when the change crosses boundaries.
- Read the complete diff and explain the result in your own words before accepting it.

A plausible answer still needs evidence

A common failure is to treat a fluent answer as proof. Choosing Technologies and Architecture requires context and judgment: verify claims against code, tests, authoritative documentation, or a reproducible observation. If the tool lacks necessary context, ask a question or gather evidence instead of filling the gap with a guess.

Use AI to make the next engineering decision easier to inspect; keep the decision and its verification yours.

Working principle

Practice

Practice: Choosing Technologies and Architecture Optional

Compare two plausible approaches with explicit criteria, evidence needs, and a decision owner.

1. Choose a small, permitted task related to Choosing Technologies and Architecture.
2. Write the goal, relevant context, constraints, and observable acceptance criteria before consulting an AI tool.
3. Request a bounded explanation, plan, or draft; note assumptions that need confirmation.
4. Verify the result with a source, focused test, reproduction, or review checklist and record what you changed or rejected.

Constraints: Use synthetic or approved information only; never include credentials, private user data, or material you are not allowed to share.

Expected outcome: A short task brief, a reviewable AI contribution, and evidence that addresses: Compare two plausible approaches with explicit criteria, evidence needs, and a decision owner.

Knowledge check

Select your answers before checking them in the course reader. The answer key is intentionally not included in this ebook.

Question 1 (required). A developer is working on Choosing Technologies and Architecture and receives a confident AI recommendation. What is the strongest next step?

- A. Apply the workflow: List requirements and nonfunctional needs; compare viable options using consistent criteria; check maintenance, deployment, and migration costs. Then verify the outcome against project evidence.
- B. Accept the recommendation because it is detailed and sounds technically plausible.
- C. Ask the model to repeat the same answer more confidently, then proceed without a project check.

Question 2 (required). A teammate asks you to act on this task: Evaluate technology choices against project constraints and team capability. Some requirements or evidence are missing. What is the strongest response?

- A. Identify the missing assumption, gather the smallest useful evidence, narrow the request, and verify affected behavior.
- B. Let the AI infer the missing requirements and make a broad change so work continues quickly.
- C. Ignore the uncertainty because the requested change appears small.

Creating a Development Roadmap

Learning objectives

- Explain how Creating a Development Roadmap supports a software-development task.
- Apply a bounded workflow that uses relevant context and makes assumptions visible.
- Evaluate and verify AI-supported work before relying on it.

Start with the engineering decision

Sequence delivery around dependencies, risk, and early feedback. The useful outcome is a decision or change that can be checked in the context where the software actually runs. AI can help you explore options and reduce drafting effort; it cannot supply missing requirements or take responsibility for the result.

This matters in planning the project with ai because plausible output can still conflict with project behavior, user expectations, or operational constraints. Begin with the observed problem and define what success looks like before requesting a solution. Keep assumptions visible so a teammate can challenge them.

A practical workflow

1. State the task in terms of the behavior or decision you need: sequence delivery around dependencies, risk, and early feedback.
2. Supply only the relevant code, contract, evidence, and constraints; remove secrets and unrelated data.
3. Identify foundational decisions and external dependencies; prioritize thin end-to-end slices; add checkpoints and revise estimates with evidence.
4. Ask for assumptions and alternatives when the evidence is incomplete; do not let a confident tone substitute for a source or reproduction.
5. Review the result in the project, run proportionate checks, and keep, revise, or reject it based on observable evidence.

Worked scenario

Deliver authentication and one protected workflow before building every page; this tests the key access boundary early. A useful request separates what is known from what is suspected. For example: “Here is the sanitized behavior, relevant contract, and constraint. First identify the assumptions and propose the smallest experiment that distinguishes the likely causes. Do not change unrelated interfaces.” This asks for reasoning that can be tested rather than a patch detached from evidence.

task-brief.txt

A compact brief; replace bracketed text with project-specific evidence.

Goal: Sequence delivery around dependencies, risk, and early feedback.

Context: [relevant behavior, files, version, or sanitized evidence]

Constraints: [invariants, policy, compatibility, and out-of-scope work]

Request: Identify foundational decisions and external dependencies; prioritize thin end-to-end slices; add checkpoints and revise estimates with evidence.

Check: [test, source, reproduction, or review that will verify the result]

Review and verification

- Compare the suggestion with the canonical requirement and nearby project conventions.
- Check normal, boundary, and failure behavior; inspect what the proposed change leaves untouched.
- Run the narrowest useful automated check, then broader checks when the change crosses boundaries.
- Read the complete diff and explain the result in your own words before accepting it.

A plausible answer still needs evidence

A common failure is to treat a fluent answer as proof. Creating a Development Roadmap requires context and judgment: verify claims against code, tests, authoritative documentation, or a reproducible observation. If the tool lacks necessary context, ask a question or gather evidence instead of filling the gap with a guess.

Use AI to make the next engineering decision easier to inspect; keep the decision and its verification yours.

Working principle

Practice

Practice: Creating a Development Roadmap Optional

Draft a short roadmap with milestones, dependencies, risk checks, and a feedback point.

1. Choose a small, permitted task related to Creating a Development Roadmap.
2. Write the goal, relevant context, constraints, and observable acceptance criteria before consulting an AI tool.
3. Request a bounded explanation, plan, or draft; note assumptions that need confirmation.
4. Verify the result with a source, focused test, reproduction, or review checklist and record what you changed or rejected.

Constraints: Use synthetic or approved information only; never include credentials, private user data, or material you are not allowed to share.

Expected outcome: A short task brief, a reviewable AI contribution, and evidence that addresses: Draft a short roadmap with milestones, dependencies, risk checks, and a feedback point.

Knowledge check

Select your answers before checking them in the course reader. The answer key is intentionally not included in this ebook.

Question 1 (required). A developer is working on Creating a Development Roadmap and receives a confident AI recommendation. What is the strongest next step?

- A. Apply the workflow: Identify foundational decisions and external dependencies; prioritize thin end-to-end slices; add checkpoints and revise estimates with evidence. Then verify the outcome against project evidence.
- B. Accept the recommendation because it is detailed and sounds technically plausible.
- C. Ask the model to repeat the same answer more confidently, then proceed without a project check.

Question 2 (required). A teammate asks you to act on this task: Sequence delivery around dependencies, risk, and early feedback. Some requirements or evidence are missing. What is the strongest response?

- A. Identify the missing assumption, gather the smallest useful evidence, narrow the request, and verify affected behavior.
- B. Let the AI infer the missing requirements and make a broad change so work continues quickly.
- C. Ignore the uncertainty because the requested change appears small.

Using AI During Implementation

Learning objectives

- Explain how Using AI During Implementation supports a software-development task.
- Apply a bounded workflow that uses relevant context and makes assumptions visible.
- Evaluate and verify AI-supported work before relying on it.

Start with the engineering decision

Keep implementation aligned with approved requirements as new questions arise. The useful outcome is a decision or change that can be checked in the context where the software actually runs. AI can help you explore options and reduce drafting effort; it cannot supply missing requirements or take responsibility for the result.

This matters in planning the project with ai because plausible output can still conflict with project behavior, user expectations, or operational constraints. Begin with the observed problem and define what success looks like before requesting a solution. Keep assumptions visible so a teammate can challenge them.

A practical workflow

1. State the task in terms of the behavior or decision you need: keep implementation aligned with approved requirements as new questions arise.
2. Supply only the relevant code, contract, evidence, and constraints; remove secrets and unrelated data.
3. Use the plan as context; ask focused questions at uncertainty points; log changed assumptions; compare code with acceptance criteria continuously.
4. Ask for assumptions and alternatives when the evidence is incomplete; do not let a confident tone substitute for a source or reproduction.
5. Review the result in the project, run proportionate checks, and keep, revise, or reject it based on observable evidence.

Worked scenario

When an API response shape differs from the plan, resolve the contract before generating client code against either version. A useful request separates what is known from what is suspected. For example: “Here is the sanitized behavior, relevant contract, and constraint. First identify the assumptions and propose the smallest experiment that distinguishes the likely causes. Do not change unrelated interfaces.” This asks for reasoning that can be tested rather than a patch detached from evidence.

task-brief.txt

A compact brief; replace bracketed text with project-specific evidence.

Goal: Keep implementation aligned with approved requirements as new questions arise.

Context: [relevant behavior, files, version, or sanitized evidence]

Constraints: [invariants, policy, compatibility, and out-of-scope work]

Request: Use the plan as context; ask focused questions at uncertainty points; log changed assumptions; compare code with acceptance criteria continuously.

Check: [test, source, reproduction, or review that will verify the result]

Review and verification

- Compare the suggestion with the canonical requirement and nearby project conventions.
- Check normal, boundary, and failure behavior; inspect what the proposed change leaves untouched.
- Run the narrowest useful automated check, then broader checks when the change crosses boundaries.
- Read the complete diff and explain the result in your own words before accepting it.

A plausible answer still needs evidence

A common failure is to treat a fluent answer as proof. Using AI During Implementation requires context and judgment: verify claims against code, tests, authoritative documentation, or a reproducible observation. If the tool lacks necessary context, ask a question or gather evidence instead of filling the gap with a guess.

Use AI to make the next engineering decision easier to inspect; keep the decision and its verification yours.

Working principle

Practice

Practice: Using AI During Implementation Optional

Choose an implementation task and define when you would ask AI, when you would consult project owners, and how to verify.

1. Choose a small, permitted task related to Using AI During Implementation.
2. Write the goal, relevant context, constraints, and observable acceptance criteria before consulting an AI tool.
3. Request a bounded explanation, plan, or draft; note assumptions that need confirmation.
4. Verify the result with a source, focused test, reproduction, or review checklist and record what you changed or rejected.

Constraints: Use synthetic or approved information only; never include credentials, private user data, or material you are not allowed to share.

Expected outcome: A short task brief, a reviewable AI contribution, and evidence that addresses: Choose an implementation task and define when you would ask AI, when you would consult project owners, and how to verify.

Knowledge check

Select your answers before checking them in the course reader. The answer key is intentionally not included in this ebook.

Question 1 (required). A developer is working on Using AI During Implementation and receives a confident AI recommendation. What is the strongest next step?

- A. Apply the workflow: Use the plan as context; ask focused questions at uncertainty points; log changed assumptions; compare code with acceptance criteria continuously. Then verify the outcome against project evidence.
- B. Accept the recommendation because it is detailed and sounds technically plausible.
- C. Ask the model to repeat the same answer more confidently, then proceed without a project check.

Question 2 (required). A teammate asks you to act on this task: Keep implementation aligned with approved requirements as new questions arise. Some requirements or evidence are missing. What is the strongest response?

- A. Identify the missing assumption, gather the smallest useful evidence, narrow the request, and verify affected behavior.
- B. Let the AI infer the missing requirements and make a broad change so work continues quickly.
- C. Ignore the uncertainty because the requested change appears small.

Generating Application Code

Learning objectives

- Explain how Generating Application Code supports a software-development task.
- Apply a bounded workflow that uses relevant context and makes assumptions visible.
- Evaluate and verify AI-supported work before relying on it.

Start with the engineering decision

Generate application slices from contracts and acceptance criteria, not from vague feature labels. The useful outcome is a decision or change that can be checked in the context where the software actually runs. AI can help you explore options and reduce drafting effort; it cannot supply missing requirements or take responsibility for the result.

This matters in building, debugging, and testing with ai because plausible output can still conflict with project behavior, user expectations, or operational constraints. Begin with the observed problem and define what success looks like before requesting a solution. Keep assumptions visible so a teammate can challenge them.

A practical workflow

1. State the task in terms of the behavior or decision you need: generate application slices from contracts and acceptance criteria, not from vague feature labels.
2. Supply only the relevant code, contract, evidence, and constraints; remove secrets and unrelated data.
3. Specify inputs, outputs, errors, authorization, and conventions; request one layer or vertical slice; inspect integration and failure behavior.
4. Ask for assumptions and alternatives when the evidence is incomplete; do not let a confident tone substitute for a source or reproduction.
5. Review the result in the project, run proportionate checks, and keep, revise, or reject it based on observable evidence.

Worked scenario

A task to create a comment endpoint needs ownership rules, validation, response shape, and persistence behavior before code generation. A useful request separates what is known from what is suspected. For example: “Here is the sanitized behavior, relevant contract, and constraint. First identify the assumptions and propose the smallest experiment that distinguishes the likely causes. Do not change unrelated interfaces.” This asks for reasoning that can be tested rather than a patch detached from evidence.

task-brief.txt

A compact brief; replace bracketed text with project-specific evidence.

Goal: Generate application slices from contracts and acceptance criteria, not from vague feature labels.

Context: [relevant behavior, files, version, or sanitized evidence]

Constraints: [invariants, policy, compatibility, and out-of-scope work]

Request: Specify inputs, outputs, errors, authorization, and conventions; request one layer or vertical slice; inspect integration and failure behavior.

Check: [test, source, reproduction, or review that will verify the result]

Review and verification

- Compare the suggestion with the canonical requirement and nearby project conventions.
- Check normal, boundary, and failure behavior; inspect what the proposed change leaves untouched.
- Run the narrowest useful automated check, then broader checks when the change crosses boundaries.
- Read the complete diff and explain the result in your own words before accepting it.

A plausible answer still needs evidence

A common failure is to treat a fluent answer as proof. Generating Application Code requires context and judgment: verify claims against code, tests, authoritative documentation, or a reproducible observation. If the tool lacks necessary context, ask a question or gather evidence instead of filling the gap with a guess.

Use AI to make the next engineering decision easier to inspect; keep the decision and its verification yours.

Working principle

Practice

Practice: Generating Application Code Optional

Write a complete implementation brief for a small application feature, including an unauthorized and invalid-input case.

1. Choose a small, permitted task related to Generating Application Code.
2. Write the goal, relevant context, constraints, and observable acceptance criteria before consulting an AI tool.
3. Request a bounded explanation, plan, or draft; note assumptions that need confirmation.
4. Verify the result with a source, focused test, reproduction, or review checklist and record what you changed or rejected.

Constraints: Use synthetic or approved information only; never include credentials, private user data, or material you are not allowed to share.

Expected outcome: A short task brief, a reviewable AI contribution, and evidence that addresses: Write a complete implementation brief for a small application feature, including an unauthorized and invalid-input case.

Knowledge check

Select your answers before checking them in the course reader. The answer key is intentionally not included in this ebook.

Question 1 (required). A developer is working on Generating Application Code and receives a confident AI recommendation. What is the strongest next step?

- A. Apply the workflow: Specify inputs, outputs, errors, authorization, and conventions; request one layer or vertical slice; inspect integration and failure behavior. Then verify the outcome against project evidence.
- B. Accept the recommendation because it is detailed and sounds technically plausible.
- C. Ask the model to repeat the same answer more confidently, then proceed without a project check.

Question 2 (required). A teammate asks you to act on this task: Generate application slices from contracts and acceptance criteria, not from vague feature labels. Some requirements or evidence are missing. What is the strongest response?

- A. Identify the missing assumption, gather the smallest useful evidence, narrow the request, and verify affected behavior.
- B. Let the AI infer the missing requirements and make a broad change so work continues quickly.
- C. Ignore the uncertainty because the requested change appears small.

Working with Frontend Development

Learning objectives

- Explain how Working with Frontend Development supports a software-development task.
- Apply a bounded workflow that uses relevant context and makes assumptions visible.
- Evaluate and verify AI-supported work before relying on it.

Start with the engineering decision

Use AI to improve UI behavior while preserving accessibility and state correctness. The useful outcome is a decision or change that can be checked in the context where the software actually runs. AI can help you explore options and reduce drafting effort; it cannot supply missing requirements or take responsibility for the result.

This matters in building, debugging, and testing with ai because plausible output can still conflict with project behavior, user expectations, or operational constraints. Begin with the observed problem and define what success looks like before requesting a solution. Keep assumptions visible so a teammate can challenge them.

A practical workflow

1. State the task in terms of the behavior or decision you need: use AI to improve UI behavior while preserving accessibility and state correctness.
2. Supply only the relevant code, contract, evidence, and constraints; remove secrets and unrelated data.
3. Describe user-visible states, data flow, responsive constraints, and component conventions; test keyboard, loading, empty, error, and success states.
4. Ask for assumptions and alternatives when the evidence is incomplete; do not let a confident tone substitute for a source or reproduction.
5. Review the result in the project, run proportionate checks, and keep, revise, or reject it based on observable evidence.

Worked scenario

A search component should handle query changes, pending requests, stale responses, no results, and keyboard submission. A useful request separates what is known from what is suspected. For example: “Here is the sanitized behavior, relevant contract, and constraint. First identify the assumptions and propose the smallest experiment that distinguishes the likely causes. Do not change unrelated interfaces.” This asks for reasoning that can be tested rather than a patch detached from evidence.

task-brief.txt

A compact brief; replace bracketed text with project-specific evidence.

Goal: Use AI to improve UI behavior while preserving accessibility and state correctness.

Context: [relevant behavior, files, version, or sanitized evidence]

Constraints: [invariants, policy, compatibility, and out-of-scope work]

Request: Describe user-visible states, data flow, responsive constraints, and component conventions; test keyboard, loading, empty, error, and success states.

Check: [test, source, reproduction, or review that will verify the result]

Review and verification

- Compare the suggestion with the canonical requirement and nearby project conventions.
- Check normal, boundary, and failure behavior; inspect what the proposed change leaves untouched.
- Run the narrowest useful automated check, then broader checks when the change crosses boundaries.
- Read the complete diff and explain the result in your own words before accepting it.

A plausible answer still needs evidence

A common failure is to treat a fluent answer as proof. Working with Frontend Development requires context and judgment: verify claims against code, tests, authoritative documentation, or a reproducible observation. If the tool lacks necessary context, ask a question or gather evidence instead of filling the gap with a guess.

Use AI to make the next engineering decision easier to inspect; keep the decision and its verification yours.

Working principle

Practice

Practice: Working with Frontend Development Optional

Review one frontend flow against state, accessibility, and responsive checks; ask AI for gaps, then verify them yourself.

1. Choose a small, permitted task related to Working with Frontend Development.
2. Write the goal, relevant context, constraints, and observable acceptance criteria before consulting an AI tool.
3. Request a bounded explanation, plan, or draft; note assumptions that need confirmation.
4. Verify the result with a source, focused test, reproduction, or review checklist and record what you changed or rejected.

Constraints: Use synthetic or approved information only; never include credentials, private user data, or material you are not allowed to share.

Expected outcome: A short task brief, a reviewable AI contribution, and evidence that addresses: Review one frontend flow against state, accessibility, and responsive checks; ask AI for gaps, then verify them yourself.

Knowledge check

Select your answers before checking them in the course reader. The answer key is intentionally not included in this ebook.

Question 1 (required). A developer is working on Working with Frontend Development and receives a confident AI recommendation. What is the strongest next step?

- A. Apply the workflow: Describe user-visible states, data flow, responsive constraints, and component conventions; test keyboard, loading, empty, error, and success states. Then verify the outcome against project evidence.
- B. Accept the recommendation because it is detailed and sounds technically plausible.
- C. Ask the model to repeat the same answer more confidently, then proceed without a project check.

Question 2 (required). A teammate asks you to act on this task: Use AI to improve UI behavior while preserving accessibility and state correctness. Some requirements or evidence are missing. What is the strongest response?

- A. Identify the missing assumption, gather the smallest useful evidence, narrow the request, and verify affected behavior.
- B. Let the AI infer the missing requirements and make a broad change so work continues quickly.
- C. Ignore the uncertainty because the requested change appears small.

Working with Backend Development

Learning objectives

- Explain how Working with Backend Development supports a software-development task.
- Apply a bounded workflow that uses relevant context and makes assumptions visible.
- Evaluate and verify AI-supported work before relying on it.

Start with the engineering decision

Design backend changes around contracts, validation, authorization, and failure handling. The useful outcome is a decision or change that can be checked in the context where the software actually runs. AI can help you explore options and reduce drafting effort; it cannot supply missing requirements or take responsibility for the result.

This matters in building, debugging, and testing with ai because plausible output can still conflict with project behavior, user expectations, or operational constraints. Begin with the observed problem and define what success looks like before requesting a solution. Keep assumptions visible so a teammate can challenge them.

A practical workflow

1. State the task in terms of the behavior or decision you need: design backend changes around contracts, validation, authorization, and failure handling.
2. Supply only the relevant code, contract, evidence, and constraints; remove secrets and unrelated data.
3. Trace request to response; validate at boundaries; enforce ownership server-side; keep business logic testable; avoid leaking internal errors.
4. Ask for assumptions and alternatives when the evidence is incomplete; do not let a confident tone substitute for a source or reproduction.
5. Review the result in the project, run proportionate checks, and keep, revise, or reject it based on observable evidence.

Worked scenario

A resource route checks authenticated ownership before returning the record and returns a consistent not-found or forbidden policy. A useful request separates what is known from what is suspected. For example: “Here is the sanitized behavior, relevant contract, and constraint. First identify the assumptions and propose the smallest experiment that distinguishes the likely causes. Do not change unrelated interfaces.” This asks for reasoning that can be tested rather than a patch detached from evidence.

task-brief.txt

A compact brief; replace bracketed text with project-specific evidence.

Goal: Design backend changes around contracts, validation, authorization, and failure handling.

Context: [relevant behavior, files, version, or sanitized evidence]

Constraints: [invariants, policy, compatibility, and out-of-scope work]

Request: Trace request to response; validate at boundaries; enforce ownership server-side; keep business logic testable; avoid leaking internal errors.

Check: [test, source, reproduction, or review that will verify the result]

Review and verification

- Compare the suggestion with the canonical requirement and nearby project conventions.
- Check normal, boundary, and failure behavior; inspect what the proposed change leaves untouched.
- Run the narrowest useful automated check, then broader checks when the change crosses boundaries.
- Read the complete diff and explain the result in your own words before accepting it.

A plausible answer still needs evidence

A common failure is to treat a fluent answer as proof. Working with Backend Development requires context and judgment: verify claims against code, tests, authoritative documentation, or a reproducible observation. If the tool lacks necessary context, ask a question or gather evidence instead of filling the gap with a guess.

Use AI to make the next engineering decision easier to inspect; keep the decision and its verification yours.

Working principle

Practice

Practice: Working with Backend Development Optional

Draft a backend endpoint contract with validation, authorization, status behavior, and focused tests.

1. Choose a small, permitted task related to Working with Backend Development.
2. Write the goal, relevant context, constraints, and observable acceptance criteria before consulting an AI tool.
3. Request a bounded explanation, plan, or draft; note assumptions that need confirmation.
4. Verify the result with a source, focused test, reproduction, or review checklist and record what you changed or rejected.

Constraints: Use synthetic or approved information only; never include credentials, private user data, or material you are not allowed to share.

Expected outcome: A short task brief, a reviewable AI contribution, and evidence that addresses: Draft a backend endpoint contract with validation, authorization, status behavior, and focused tests.

Knowledge check

Select your answers before checking them in the course reader. The answer key is intentionally not included in this ebook.

Question 1 (required). A developer is working on Working with Backend Development and receives a confident AI recommendation. What is the strongest next step?

- A. Apply the workflow: Trace request to response; validate at boundaries; enforce ownership server-side; keep business logic testable; avoid leaking internal errors. Then verify the outcome against project evidence.
- B. Accept the recommendation because it is detailed and sounds technically plausible.
- C. Ask the model to repeat the same answer more confidently, then proceed without a project check.

Question 2 (required). A teammate asks you to act on this task: Design backend changes around contracts, validation, authorization, and failure handling. Some requirements or evidence are missing. What is the strongest response?

- A. Identify the missing assumption, gather the smallest useful evidence, narrow the request, and verify affected behavior.
- B. Let the AI infer the missing requirements and make a broad change so work continues quickly.
- C. Ignore the uncertainty because the requested change appears small.

Working with Databases

Learning objectives

- Explain how Working with Databases supports a software-development task.
- Apply a bounded workflow that uses relevant context and makes assumptions visible.
- Evaluate and verify AI-supported work before relying on it.

Start with the engineering decision

Use AI to reason about data shape and queries while checking integrity and operational impact. The useful outcome is a decision or change that can be checked in the context where the software actually runs. AI can help you explore options and reduce drafting effort; it cannot supply missing requirements or take responsibility for the result.

This matters in building, debugging, and testing with ai because plausible output can still conflict with project behavior, user expectations, or operational constraints. Begin with the observed problem and define what success looks like before requesting a solution. Keep assumptions visible so a teammate can challenge them.

A practical workflow

1. State the task in terms of the behavior or decision you need: use AI to reason about data shape and queries while checking integrity and operational impact.
2. Supply only the relevant code, contract, evidence, and constraints; remove secrets and unrelated data.
3. Share schemas and access patterns, not real customer data; examine indexes, constraints, migrations, and transaction needs; test representative edge cases.
4. Ask for assumptions and alternatives when the evidence is incomplete; do not let a confident tone substitute for a source or reproduction.
5. Review the result in the project, run proportionate checks, and keep, revise, or reject it based on observable evidence.

Worked scenario

A uniqueness requirement must be enforced by a database constraint when concurrent requests could bypass an application-only check. A useful request separates what is known from what is suspected. For example: “Here is the sanitized behavior, relevant contract, and constraint. First identify the assumptions and propose the smallest experiment that distinguishes the likely causes. Do not change unrelated interfaces.” This asks for reasoning that can be tested rather than a patch detached from evidence.

task-brief.txt

A compact brief; replace bracketed text with project-specific evidence.

Goal: Use AI to reason about data shape and queries while checking integrity and operational impact.

Context: [relevant behavior, files, version, or sanitized evidence]

Constraints: [invariants, policy, compatibility, and out-of-scope work]

Request: Share schemas and access patterns, not real customer data; examine indexes, constraints, migrations, and transaction needs; test representative edge cases.

Check: [test, source, reproduction, or review that will verify the result]

Review and verification

- Compare the suggestion with the canonical requirement and nearby project conventions.
- Check normal, boundary, and failure behavior; inspect what the proposed change leaves untouched.
- Run the narrowest useful automated check, then broader checks when the change crosses boundaries.
- Read the complete diff and explain the result in your own words before accepting it.

A plausible answer still needs evidence

A common failure is to treat a fluent answer as proof. Working with Databases requires context and judgment: verify claims against code, tests, authoritative documentation, or a reproducible observation. If the tool lacks necessary context, ask a question or gather evidence instead of filling the gap with a guess.

Use AI to make the next engineering decision easier to inspect; keep the decision and its verification yours.

Working principle

Practice

Practice: Working with Databases Optional

Analyze a small data model and list integrity rules, query patterns, and safe migration questions.

1. Choose a small, permitted task related to Working with Databases.
2. Write the goal, relevant context, constraints, and observable acceptance criteria before consulting an AI tool.
3. Request a bounded explanation, plan, or draft; note assumptions that need confirmation.
4. Verify the result with a source, focused test, reproduction, or review checklist and record what you changed or rejected.

Constraints: Use synthetic or approved information only; never include credentials, private user data, or material you are not allowed to share.

Expected outcome: A short task brief, a reviewable AI contribution, and evidence that addresses: Analyze a small data model and list integrity rules, query patterns, and safe migration questions.

Knowledge check

Select your answers before checking them in the course reader. The answer key is intentionally not included in this ebook.

Question 1 (required). A developer is working on Working with Databases and receives a confident AI recommendation. What is the strongest next step?

- A. Apply the workflow: Share schemas and access patterns, not real customer data; examine indexes, constraints, migrations, and transaction needs; test representative edge cases. Then verify the outcome against project evidence.
- B. Accept the recommendation because it is detailed and sounds technically plausible.
- C. Ask the model to repeat the same answer more confidently, then proceed without a project check.

Question 2 (required). A teammate asks you to act on this task: Use AI to reason about data shape and queries while checking integrity and operational impact Some requirements or evidence are missing. What is the strongest response?

- A. Identify the missing assumption, gather the smallest useful evidence, narrow the request, and verify affected behavior.
- B. Let the AI infer the missing requirements and make a broad change so work continues quickly.
- C. Ignore the uncertainty because the requested change appears small.

API Integration and Testing

Learning objectives

- Explain how API Integration and Testing supports a software-development task.
- Apply a bounded workflow that uses relevant context and makes assumptions visible.
- Evaluate and verify AI-supported work before relying on it.

Start with the engineering decision

Integrate APIs by verifying contracts, failure modes, and boundary behavior. The useful outcome is a decision or change that can be checked in the context where the software actually runs. AI can help you explore options and reduce drafting effort; it cannot supply missing requirements or take responsibility for the result.

This matters in building, debugging, and testing with ai because plausible output can still conflict with project behavior, user expectations, or operational constraints. Begin with the observed problem and define what success looks like before requesting a solution. Keep assumptions visible so a teammate can challenge them.

A practical workflow

1. State the task in terms of the behavior or decision you need: integrate APIs by verifying contracts, failure modes, and boundary behavior.
2. Supply only the relevant code, contract, evidence, and constraints; remove secrets and unrelated data.
3. Confirm method, authentication, headers, payload, response, and error contract; isolate network failures; test success and failure paths.
4. Ask for assumptions and alternatives when the evidence is incomplete; do not let a confident tone substitute for a source or reproduction.
5. Review the result in the project, run proportionate checks, and keep, revise, or reject it based on observable evidence.

Worked scenario

A payment provider timeout is an unknown outcome; avoid blindly repeating a non-idempotent action and follow the provider's reconciliation contract. A useful request separates what is known from what is suspected. For example: "Here is the sanitized behavior, relevant contract, and constraint. First identify the assumptions and propose the smallest experiment that distinguishes the likely causes. Do not change unrelated interfaces." This asks for reasoning that can be tested rather than a patch detached from evidence.

task-brief.txt

A compact brief; replace bracketed text with project-specific evidence.

Goal: Integrate APIs by verifying contracts, failure modes, and boundary behavior.

Context: [relevant behavior, files, version, or sanitized evidence]

Constraints: [invariants, policy, compatibility, and out-of-scope work]

Request: Confirm method, authentication, headers, payload, response, and error contract; isolate network failures; test success and failure paths.

Check: [test, source, reproduction, or review that will verify the result]

Review and verification

- Compare the suggestion with the canonical requirement and nearby project conventions.
- Check normal, boundary, and failure behavior; inspect what the proposed change leaves untouched.
- Run the narrowest useful automated check, then broader checks when the change crosses boundaries.
- Read the complete diff and explain the result in your own words before accepting it.

A plausible answer still needs evidence

A common failure is to treat a fluent answer as proof. API Integration and Testing requires context and judgment: verify claims against code, tests, authoritative documentation, or a reproducible observation. If the tool lacks necessary context, ask a question or gather evidence instead of filling the gap with a guess.

Use AI to make the next engineering decision easier to inspect; keep the decision and its verification yours.

Working principle

Practice

Practice: API Integration and Testing Optional

Create an API integration test matrix covering success, invalid input, unauthorized access, timeout, and retry behavior.

1. Choose a small, permitted task related to API Integration and Testing.
2. Write the goal, relevant context, constraints, and observable acceptance criteria before consulting an AI tool.
3. Request a bounded explanation, plan, or draft; note assumptions that need confirmation.
4. Verify the result with a source, focused test, reproduction, or review checklist and record what you changed or rejected.

Constraints: Use synthetic or approved information only; never include credentials, private user data, or material you are not allowed to share.

Expected outcome: A short task brief, a reviewable AI contribution, and evidence that addresses: Create an API integration test matrix covering success, invalid input, unauthorized access, timeout, and retry behavior.

Knowledge check

Select your answers before checking them in the course reader. The answer key is intentionally not included in this ebook.

Question 1 (required). A developer is working on API Integration and Testing and receives a confident AI recommendation. What is the strongest next step?

- A. Apply the workflow: Confirm method, authentication, headers, payload, response, and error contract; isolate network failures; test success and failure paths. Then verify the outcome against project evidence.
- B. Accept the recommendation because it is detailed and sounds technically plausible.
- C. Ask the model to repeat the same answer more confidently, then proceed without a project check.

Question 2 (required). A teammate asks you to act on this task: Integrate APIs by verifying contracts, failure modes, and boundary behavior Some requirements or evidence are missing. What is the strongest response?

- A. Identify the missing assumption, gather the smallest useful evidence, narrow the request, and verify affected behavior.
- B. Let the AI infer the missing requirements and make a broad change so work continues quickly.
- C. Ignore the uncertainty because the requested change appears small.

Debugging Complex Problems

Learning objectives

- Explain how Debugging Complex Problems supports a software-development task.
- Apply a bounded workflow that uses relevant context and makes assumptions visible.
- Evaluate and verify AI-supported work before relying on it.

Start with the engineering decision

Investigate cross-layer failures through timelines, boundaries, and falsifiable hypotheses. The useful outcome is a decision or change that can be checked in the context where the software actually runs. AI can help you explore options and reduce drafting effort; it cannot supply missing requirements or take responsibility for the result.

This matters in building, debugging, and testing with ai because plausible output can still conflict with project behavior, user expectations, or operational constraints. Begin with the observed problem and define what success looks like before requesting a solution. Keep assumptions visible so a teammate can challenge them.

A practical workflow

1. State the task in terms of the behavior or decision you need: investigate cross-layer failures through timelines, boundaries, and falsifiable hypotheses.
2. Supply only the relevant code, contract, evidence, and constraints; remove secrets and unrelated data.
3. Reproduce; collect sanitized observations; trace identifiers across layers; rank causes; change one variable; verify with a regression test.
4. Ask for assumptions and alternatives when the evidence is incomplete; do not let a confident tone substitute for a source or reproduction.
5. Review the result in the project, run proportionate checks, and keep, revise, or reject it based on observable evidence.

Worked scenario

If saved data appears stale, compare browser payload, API response, database read, and cache behavior using one correlated request. A useful request separates what is known from what is suspected. For example: “Here is the sanitized behavior, relevant contract, and constraint. First identify the assumptions and propose the smallest experiment that distinguishes the likely causes. Do not change unrelated interfaces.” This asks for reasoning that can be tested rather than a patch detached from evidence.

task-brief.txt

A compact brief; replace bracketed text with project-specific evidence.

Goal: Investigate cross-layer failures through timelines, boundaries, and falsifiable hypotheses.

Context: [relevant behavior, files, version, or sanitized evidence]

Constraints: [invariants, policy, compatibility, and out-of-scope work]

Request: Reproduce; collect sanitized observations; trace identifiers across layers; rank causes; change one variable; verify with a regression test.

Check: [test, source, reproduction, or review that will verify the result]

Review and verification

- Compare the suggestion with the canonical requirement and nearby project conventions.
- Check normal, boundary, and failure behavior; inspect what the proposed change leaves untouched.
- Run the narrowest useful automated check, then broader checks when the change crosses boundaries.
- Read the complete diff and explain the result in your own words before accepting it.

A plausible answer still needs evidence

A common failure is to treat a fluent answer as proof. Debugging Complex Problems requires context and judgment: verify claims against code, tests, authoritative documentation, or a reproducible observation. If the tool lacks necessary context, ask a question or gather evidence instead of filling the gap with a guess.

Use AI to make the next engineering decision easier to inspect; keep the decision and its verification yours.

Working principle

Practice

Practice: Debugging Complex Problems Optional

Build a hypothesis table for a multi-layer defect with evidence that would confirm or reject each cause.

1. Choose a small, permitted task related to Debugging Complex Problems.
2. Write the goal, relevant context, constraints, and observable acceptance criteria before consulting an AI tool.
3. Request a bounded explanation, plan, or draft; note assumptions that need confirmation.
4. Verify the result with a source, focused test, reproduction, or review checklist and record what you changed or rejected.

Constraints: Use synthetic or approved information only; never include credentials, private user data, or material you are not allowed to share.

Expected outcome: A short task brief, a reviewable AI contribution, and evidence that addresses: Build a hypothesis table for a multi-layer defect with evidence that would confirm or reject each cause.

Knowledge check

Select your answers before checking them in the course reader. The answer key is intentionally not included in this ebook.

Question 1 (required). A developer is working on Debugging Complex Problems and receives a confident AI recommendation. What is the strongest next step?

- A. Apply the workflow: Reproduce; collect sanitized observations; trace identifiers across layers; rank causes; change one variable; verify with a regression test. Then verify the outcome against project evidence.
- B. Accept the recommendation because it is detailed and sounds technically plausible.
- C. Ask the model to repeat the same answer more confidently, then proceed without a project check.

Question 2 (required). A teammate asks you to act on this task: Investigate cross-layer failures through timelines, boundaries, and falsifiable hypotheses. Some requirements or evidence are missing. What is the strongest response?

- A. Identify the missing assumption, gather the smallest useful evidence, narrow the request, and verify affected behavior.
- B. Let the AI infer the missing requirements and make a broad change so work continues quickly.
- C. Ignore the uncertainty because the requested change appears small.

Automated Testing and Quality Assurance

Learning objectives

- Explain how Automated Testing and Quality Assurance supports a software-development task.
- Apply a bounded workflow that uses relevant context and makes assumptions visible.
- Evaluate and verify AI-supported work before relying on it.

Start with the engineering decision

Use tests as executable evidence across unit, integration, and user-visible behavior. The useful outcome is a decision or change that can be checked in the context where the software actually runs. AI can help you explore options and reduce drafting effort; it cannot supply missing requirements or take responsibility for the result.

This matters in building, debugging, and testing with ai because plausible output can still conflict with project behavior, user expectations, or operational constraints. Begin with the observed problem and define what success looks like before requesting a solution. Keep assumptions visible so a teammate can challenge them.

A practical workflow

1. State the task in terms of the behavior or decision you need: use tests as executable evidence across unit, integration, and user-visible behavior.
2. Supply only the relevant code, contract, evidence, and constraints; remove secrets and unrelated data.
3. Choose test level by risk; cover meaningful partitions and boundaries; review generated tests for assertions and isolation; run them in the normal workflow.
4. Ask for assumptions and alternatives when the evidence is incomplete; do not let a confident tone substitute for a source or reproduction.
5. Review the result in the project, run proportionate checks, and keep, revise, or reject it based on observable evidence.

Worked scenario

A mocked unit test cannot prove a route enforces database-backed ownership; add an integration test at that boundary. A useful request separates what is known from what is suspected. For example: "Here is the sanitized behavior, relevant contract, and constraint. First identify the assumptions and propose the smallest experiment that distinguishes the likely causes. Do not change unrelated interfaces." This asks for reasoning that can be tested rather than a patch detached from evidence.

task-brief.txt

A compact brief; replace bracketed text with project-specific evidence.

Goal: Use tests as executable evidence across unit, integration, and user-visible behavior.

Context: [relevant behavior, files, version, or sanitized evidence]

Constraints: [invariants, policy, compatibility, and out-of-scope work]

Request: Choose test level by risk; cover meaningful partitions and boundaries; review generated tests for assertions and isolation; run them in the normal workflow.

Check: [test, source, reproduction, or review that will verify the result]

Review and verification

- Compare the suggestion with the canonical requirement and nearby project conventions.
- Check normal, boundary, and failure behavior; inspect what the proposed change leaves untouched.
- Run the narrowest useful automated check, then broader checks when the change crosses boundaries.
- Read the complete diff and explain the result in your own words before accepting it.

A plausible answer still needs evidence

A common failure is to treat a fluent answer as proof. Automated Testing and Quality Assurance requires context and judgment: verify claims against code, tests, authoritative documentation, or a reproducible observation. If the tool lacks necessary context, ask a question or gather evidence instead of filling the gap with a guess.

Use AI to make the next engineering decision easier to inspect; keep the decision and its verification yours.

Working principle

Practice

Practice: Automated Testing and Quality Assurance Optional

Select a feature and propose layered tests with the risk each test detects and any behavior still requiring manual review.

1. Choose a small, permitted task related to Automated Testing and Quality Assurance.
2. Write the goal, relevant context, constraints, and observable acceptance criteria before consulting an AI tool.
3. Request a bounded explanation, plan, or draft; note assumptions that need confirmation.
4. Verify the result with a source, focused test, reproduction, or review checklist and record what you changed or rejected.

Constraints: Use synthetic or approved information only; never include credentials, private user data, or material you are not allowed to share.

Expected outcome: A short task brief, a reviewable AI contribution, and evidence that addresses: Select a feature and propose layered tests with the risk each test detects and any behavior still requiring manual review.

Knowledge check

Select your answers before checking them in the course reader. The answer key is intentionally not included in this ebook.

Question 1 (required). A developer is working on Automated Testing and Quality Assurance and receives a confident AI recommendation. What is the strongest next step?

- A. Apply the workflow: Choose test level by risk; cover meaningful partitions and boundaries; review generated tests for assertions and isolation; run them in the normal workflow. Then verify the outcome against project evidence.
- B. Accept the recommendation because it is detailed and sounds technically plausible.
- C. Ask the model to repeat the same answer more confidently, then proceed without a project check.

Question 2 (required). A teammate asks you to act on this task: Use tests as executable evidence across unit, integration, and user-visible behavior. Some requirements or evidence are missing. What is the strongest response?

- A. Identify the missing assumption, gather the smallest useful evidence, narrow the request, and verify affected behavior.
- B. Let the AI infer the missing requirements and make a broad change so work continues quickly.
- C. Ignore the uncertainty because the requested change appears small.

Code Review with AI

Learning objectives

- Explain how Code Review with AI supports a software-development task.
- Apply a bounded workflow that uses relevant context and makes assumptions visible.
- Evaluate and verify AI-supported work before relying on it.

Start with the engineering decision

Use AI as a reviewer that proposes risks, while humans decide severity and correctness. The useful outcome is a decision or change that can be checked in the context where the software actually runs. AI can help you explore options and reduce drafting effort; it cannot supply missing requirements or take responsibility for the result.

This matters in professional ai-assisted software engineering because plausible output can still conflict with project behavior, user expectations, or operational constraints. Begin with the observed problem and define what success looks like before requesting a solution. Keep assumptions visible so a teammate can challenge them.

A practical workflow

1. State the task in terms of the behavior or decision you need: use AI as a reviewer that proposes risks, while humans decide severity and correctness.
2. Supply only the relevant code, contract, evidence, and constraints; remove secrets and unrelated data.
3. Provide a focused diff and relevant contract; ask for concrete failure modes; reproduce findings; review security and maintainability in project context.
4. Ask for assumptions and alternatives when the evidence is incomplete; do not let a confident tone substitute for a source or reproduction.
5. Review the result in the project, run proportionate checks, and keep, revise, or reject it based on observable evidence.

Worked scenario

A reviewer flags missing authorization; confirm whether middleware or the handler enforces it before reporting a defect. A useful request separates what is known from what is suspected. For example: “Here is the sanitized behavior, relevant contract, and constraint. First identify the assumptions and propose the smallest experiment that distinguishes the likely causes. Do not change unrelated interfaces.” This asks for reasoning that can be tested rather than a patch detached from evidence.

task-brief.txt

A compact brief; replace bracketed text with project-specific evidence.

Goal: Use AI as a reviewer that proposes risks, while humans decide severity and correctness.

Context: [relevant behavior, files, version, or sanitized evidence]

Constraints: [invariants, policy, compatibility, and out-of-scope work]

Request: Provide a focused diff and relevant contract; ask for concrete failure modes; reproduce findings; review security and maintainability in project context.

Check: [test, source, reproduction, or review that will verify the result]

Review and verification

- Compare the suggestion with the canonical requirement and nearby project conventions.
- Check normal, boundary, and failure behavior; inspect what the proposed change leaves untouched.
- Run the narrowest useful automated check, then broader checks when the change crosses boundaries.
- Read the complete diff and explain the result in your own words before accepting it.

A plausible answer still needs evidence

A common failure is to treat a fluent answer as proof. Code Review with AI requires context and judgment: verify claims against code, tests, authoritative documentation, or a reproducible observation. If the tool lacks necessary context, ask a question or gather evidence instead of filling the gap with a guess.

Use AI to make the next engineering decision easier to inspect; keep the decision and its verification yours.

Working principle

Practice

Practice: Code Review with AI Optional

Review a small change with an AI checklist and classify findings by evidence, impact, and confidence.

1. Choose a small, permitted task related to Code Review with AI.
2. Write the goal, relevant context, constraints, and observable acceptance criteria before consulting an AI tool.
3. Request a bounded explanation, plan, or draft; note assumptions that need confirmation.
4. Verify the result with a source, focused test, reproduction, or review checklist and record what you changed or rejected.

Constraints: Use synthetic or approved information only; never include credentials, private user data, or material you are not allowed to share.

Expected outcome: A short task brief, a reviewable AI contribution, and evidence that addresses: Review a small change with an AI checklist and classify findings by evidence, impact, and confidence.

Knowledge check

Select your answers before checking them in the course reader. The answer key is intentionally not included in this ebook.

Question 1 (required). A developer is working on Code Review with AI and receives a confident AI recommendation. What is the strongest next step?

- A. Apply the workflow: Provide a focused diff and relevant contract; ask for concrete failure modes; reproduce findings; review security and maintainability in project context. Then verify the outcome against project evidence.
- B. Accept the recommendation because it is detailed and sounds technically plausible.
- C. Ask the model to repeat the same answer more confidently, then proceed without a project check.

Question 2 (required). A teammate asks you to act on this task: Use AI as a reviewer that proposes risks, while humans decide severity and correctness. Some requirements or evidence are missing. What is the strongest response?

- A. Identify the missing assumption, gather the smallest useful evidence, narrow the request, and verify affected behavior.
- B. Let the AI infer the missing requirements and make a broad change so work continues quickly.
- C. Ignore the uncertainty because the requested change appears small.

Documentation with AI

Learning objectives

- Explain how Documentation with AI supports a software-development task.
- Apply a bounded workflow that uses relevant context and makes assumptions visible.
- Evaluate and verify AI-supported work before relying on it.

Start with the engineering decision

Draft documentation from verified behavior and keep examples executable or clearly illustrative. The useful outcome is a decision or change that can be checked in the context where the software actually runs. AI can help you explore options and reduce drafting effort; it cannot supply missing requirements or take responsibility for the result.

This matters in professional ai-assisted software engineering because plausible output can still conflict with project behavior, user expectations, or operational constraints. Begin with the observed problem and define what success looks like before requesting a solution. Keep assumptions visible so a teammate can challenge them.

A practical workflow

1. State the task in terms of the behavior or decision you need: draft documentation from verified behavior and keep examples executable or clearly illustrative.
2. Supply only the relevant code, contract, evidence, and constraints; remove secrets and unrelated data.
3. Identify audience and source files; ask for a structured draft; verify commands, defaults, and limitations against implementation; update alongside code.
4. Ask for assumptions and alternatives when the evidence is incomplete; do not let a confident tone substitute for a source or reproduction.
5. Review the result in the project, run proportionate checks, and keep, revise, or reject it based on observable evidence.

Worked scenario

A setup guide should distinguish required environment variables from optional ones and must not paste actual credentials. A useful request separates what is known from what is suspected. For example: “Here is the sanitized behavior, relevant contract, and constraint. First identify the assumptions and propose the smallest experiment that distinguishes the likely causes. Do not change unrelated interfaces.” This asks for reasoning that can be tested rather than a patch detached from evidence.

task-brief.txt

A compact brief; replace bracketed text with project-specific evidence.

Goal: Draft documentation from verified behavior and keep examples executable or clearly illustrative.

Context: [relevant behavior, files, version, or sanitized evidence]

Constraints: [invariants, policy, compatibility, and out-of-scope work]

Request: Identify audience and source files; ask for a structured draft; verify commands, defaults, and limitations against implementation; update alongside code.

Check: [test, source, reproduction, or review that will verify the result]

Review and verification

- Compare the suggestion with the canonical requirement and nearby project conventions.
- Check normal, boundary, and failure behavior; inspect what the proposed change leaves untouched.
- Run the narrowest useful automated check, then broader checks when the change crosses boundaries.
- Read the complete diff and explain the result in your own words before accepting it.

A plausible answer still needs evidence

A common failure is to treat a fluent answer as proof. Documentation with AI requires context and judgment: verify claims against code, tests, authoritative documentation, or a reproducible observation. If the tool lacks necessary context, ask a question or gather evidence instead of filling the gap with a guess.

Use AI to make the next engineering decision easier to inspect; keep the decision and its verification yours.

Working principle

Practice

Practice: Documentation with AI Optional

Improve one README section using implementation evidence and verify every command or behavior claim.

1. Choose a small, permitted task related to Documentation with AI.
2. Write the goal, relevant context, constraints, and observable acceptance criteria before consulting an AI tool.
3. Request a bounded explanation, plan, or draft; note assumptions that need confirmation.
4. Verify the result with a source, focused test, reproduction, or review checklist and record what you changed or rejected.

Constraints: Use synthetic or approved information only; never include credentials, private user data, or material you are not allowed to share.

Expected outcome: A short task brief, a reviewable AI contribution, and evidence that addresses: Improve one README section using implementation evidence and verify every command or behavior claim.

Knowledge check

Select your answers before checking them in the course reader. The answer key is intentionally not included in this ebook.

Question 1 (required). A developer is working on Documentation with AI and receives a confident AI recommendation. What is the strongest next step?

- A. Apply the workflow: Identify audience and source files; ask for a structured draft; verify commands, defaults, and limitations against implementation; update alongside code. Then verify the outcome against project evidence.
- B. Accept the recommendation because it is detailed and sounds technically plausible.
- C. Ask the model to repeat the same answer more confidently, then proceed without a project check.

Question 2 (required). A teammate asks you to act on this task: Draft documentation from verified behavior and keep examples executable or clearly illustrative Some requirements or evidence are missing. What is the strongest response?

- A. Identify the missing assumption, gather the smallest useful evidence, narrow the request, and verify affected behavior.
- B. Let the AI infer the missing requirements and make a broad change so work continues quickly.
- C. Ignore the uncertainty because the requested change appears small.

Security, Privacy, and Responsible AI Use

Learning objectives

- Explain how Security, Privacy, and Responsible AI Use supports a software-development task.
- Apply a bounded workflow that uses relevant context and makes assumptions visible.
- Evaluate and verify AI-supported work before relying on it.

Start with the engineering decision

Apply data minimization, threat awareness, and organizational policy to AI-assisted work. The useful outcome is a decision or change that can be checked in the context where the software actually runs. AI can help you explore options and reduce drafting effort; it cannot supply missing requirements or take responsibility for the result.

This matters in professional ai-assisted software engineering because plausible output can still conflict with project behavior, user expectations, or operational constraints. Begin with the observed problem and define what success looks like before requesting a solution. Keep assumptions visible so a teammate can challenge them.

A practical workflow

1. State the task in terms of the behavior or decision you need: apply data minimization, threat awareness, and organizational policy to AI-assisted work.
2. Supply only the relevant code, contract, evidence, and constraints; remove secrets and unrelated data.
3. Classify information; use approved tools; redact secrets and personal data; review generated code for injection, authorization, and unsafe defaults.
4. Ask for assumptions and alternatives when the evidence is incomplete; do not let a confident tone substitute for a source or reproduction.
5. Review the result in the project, run proportionate checks, and keep, revise, or reject it based on observable evidence.

Worked scenario

A prompt containing a production dump should be stopped and sanitized; model output does not authorize processing restricted data. A useful request separates what is known from what is suspected. For example: “Here is the sanitized behavior, relevant contract, and constraint. First identify the assumptions and propose the smallest experiment that distinguishes the likely causes. Do not change unrelated interfaces.” This asks for reasoning that can be tested rather than a patch detached from evidence.

task-brief.txt

A compact brief; replace bracketed text with project-specific evidence.

Goal: Apply data minimization, threat awareness, and organizational policy to AI-assisted work.

Context: [relevant behavior, files, version, or sanitized evidence]

Constraints: [invariants, policy, compatibility, and out-of-scope work]

Request: Classify information; use approved tools; redact secrets and personal data; review generated code for injection, authorization, and unsafe defaults.

Check: [test, source, reproduction, or review that will verify the result]

Review and verification

- Compare the suggestion with the canonical requirement and nearby project conventions.
- Check normal, boundary, and failure behavior; inspect what the proposed change leaves untouched.
- Run the narrowest useful automated check, then broader checks when the change crosses boundaries.
- Read the complete diff and explain the result in your own words before accepting it.

A plausible answer still needs evidence

A common failure is to treat a fluent answer as proof. Security, Privacy, and Responsible AI Use requires context and judgment: verify claims against code, tests, authoritative documentation, or a reproducible observation. If the tool lacks necessary context, ask a question or gather evidence instead of filling the gap with a guess.

Use AI to make the next engineering decision easier to inspect; keep the decision and its verification yours.

Working principle

Practice

Practice: Security, Privacy, and Responsible AI Use Optional

Create a safe-context checklist for an AI task and identify one security review appropriate to its code.

1. Choose a small, permitted task related to Security, Privacy, and Responsible AI Use.
2. Write the goal, relevant context, constraints, and observable acceptance criteria before consulting an AI tool.
3. Request a bounded explanation, plan, or draft; note assumptions that need confirmation.
4. Verify the result with a source, focused test, reproduction, or review checklist and record what you changed or rejected.

Constraints: Use synthetic or approved information only; never include credentials, private user data, or material you are not allowed to share.

Expected outcome: A short task brief, a reviewable AI contribution, and evidence that addresses: Create a safe-context checklist for an AI task and identify one security review appropriate to its code.

Knowledge check

Select your answers before checking them in the course reader. The answer key is intentionally not included in this ebook.

Question 1 (required). A developer is working on Security, Privacy, and Responsible AI Use and receives a confident AI recommendation. What is the strongest next step?

- A. Apply the workflow: Classify information; use approved tools; redact secrets and personal data; review generated code for injection, authorization, and unsafe defaults. Then verify the outcome against project evidence.
- B. Accept the recommendation because it is detailed and sounds technically plausible.
- C. Ask the model to repeat the same answer more confidently, then proceed without a project check.

Question 2 (required). A teammate asks you to act on this task: Apply data minimization, threat awareness, and organizational policy to AI-assisted work. Some requirements or evidence are missing. What is the strongest response?

- A. Identify the missing assumption, gather the smallest useful evidence, narrow the request, and verify affected behavior.
- B. Let the AI infer the missing requirements and make a broad change so work continues quickly.
- C. Ignore the uncertainty because the requested change appears small.

Git, Version Control, and AI-Assisted Workflows

Learning objectives

- Explain how Git, Version Control, and AI-Assisted Workflows supports a software-development task.
- Apply a bounded workflow that uses relevant context and makes assumptions visible.
- Evaluate and verify AI-supported work before relying on it.

Start with the engineering decision

Use version control to preserve a clear baseline and make AI changes reviewable. The useful outcome is a decision or change that can be checked in the context where the software actually runs. AI can help you explore options and reduce drafting effort; it cannot supply missing requirements or take responsibility for the result.

This matters in professional ai-assisted software engineering because plausible output can still conflict with project behavior, user expectations, or operational constraints. Begin with the observed problem and define what success looks like before requesting a solution. Keep assumptions visible so a teammate can challenge them.

A practical workflow

1. State the task in terms of the behavior or decision you need: use version control to preserve a clear baseline and make AI changes reviewable.
2. Supply only the relevant code, contract, evidence, and constraints; remove secrets and unrelated data.
3. Check branch and worktree; understand existing edits; isolate a bounded task; inspect diff; run checks; stage only intended files.
4. Ask for assumptions and alternatives when the evidence is incomplete; do not let a confident tone substitute for a source or reproduction.
5. Review the result in the project, run proportionate checks, and keep, revise, or reject it based on observable evidence.

Worked scenario

When the worktree has unrelated edits, record the baseline and avoid broad staging so a focused change does not capture them. A useful request separates what is known from what is suspected. For example: “Here is the sanitized behavior, relevant contract, and constraint. First identify the assumptions and propose the smallest experiment that distinguishes the likely causes. Do not change unrelated interfaces.” This asks for reasoning that can be tested rather than a patch detached from evidence.

task-brief.txt

A compact brief; replace bracketed text with project-specific evidence.

Goal: Use version control to preserve a clear baseline and make AI changes reviewable.

Context: [relevant behavior, files, version, or sanitized evidence]

Constraints: [invariants, policy, compatibility, and out-of-scope work]

Request: Check branch and worktree; understand existing edits; isolate a bounded task; inspect diff; run checks; stage only intended files.

Check: [test, source, reproduction, or review that will verify the result]

Review and verification

- Compare the suggestion with the canonical requirement and nearby project conventions.
- Check normal, boundary, and failure behavior; inspect what the proposed change leaves untouched.
- Run the narrowest useful automated check, then broader checks when the change crosses boundaries.
- Read the complete diff and explain the result in your own words before accepting it.

A plausible answer still needs evidence

A common failure is to treat a fluent answer as proof. Git, Version Control, and AI-Assisted Workflows requires context and judgment: verify claims against code, tests, authoritative documentation, or a reproducible observation. If the tool lacks necessary context, ask a question or gather evidence instead of filling the gap with a guess.

Use AI to make the next engineering decision easier to inspect; keep the decision and its verification yours.

Working principle

Practice

Practice: Git, Version Control, and AI-Assisted Workflows Optional

Write a preflight and post-change Git checklist for a task with unrelated local modifications.

1. Choose a small, permitted task related to Git, Version Control, and AI-Assisted Workflows.
2. Write the goal, relevant context, constraints, and observable acceptance criteria before consulting an AI tool.
3. Request a bounded explanation, plan, or draft; note assumptions that need confirmation.
4. Verify the result with a source, focused test, reproduction, or review checklist and record what you changed or rejected.

Constraints: Use synthetic or approved information only; never include credentials, private user data, or material you are not allowed to share.

Expected outcome: A short task brief, a reviewable AI contribution, and evidence that addresses: Write a preflight and post-change Git checklist for a task with unrelated local modifications.

Knowledge check

Select your answers before checking them in the course reader. The answer key is intentionally not included in this ebook.

Question 1 (required). A developer is working on Git, Version Control, and AI-Assisted Workflows and receives a confident AI recommendation. What is the strongest next step?

- A. Apply the workflow: Check branch and worktree; understand existing edits; isolate a bounded task; inspect diff; run checks; stage only intended files. Then verify the outcome against project evidence.
- B. Accept the recommendation because it is detailed and sounds technically plausible.
- C. Ask the model to repeat the same answer more confidently, then proceed without a project check.

Question 2 (required). A teammate asks you to act on this task: Use version control to preserve a clear baseline and make AI changes reviewable. Some requirements or evidence are missing. What is the strongest response?

- A. Identify the missing assumption, gather the smallest useful evidence, narrow the request, and verify affected behavior.
- B. Let the AI infer the missing requirements and make a broad change so work continues quickly.
- C. Ignore the uncertainty because the requested change appears small.

Maintaining and Improving AI-Assisted Codebases

Learning objectives

- Explain how Maintaining and Improving AI-Assisted Codebases supports a software-development task.
- Apply a bounded workflow that uses relevant context and makes assumptions visible.
- Evaluate and verify AI-supported work before relying on it.

Start with the engineering decision

Maintain AI-assisted systems through ownership, observability, and evidence-based cleanup. The useful outcome is a decision or change that can be checked in the context where the software actually runs. AI can help you explore options and reduce drafting effort; it cannot supply missing requirements or take responsibility for the result.

This matters in professional ai-assisted software engineering because plausible output can still conflict with project behavior, user expectations, or operational constraints. Begin with the observed problem and define what success looks like before requesting a solution. Keep assumptions visible so a teammate can challenge them.

A practical workflow

1. State the task in terms of the behavior or decision you need: maintain AI-assisted systems through ownership, observability, and evidence-based cleanup.
2. Supply only the relevant code, contract, evidence, and constraints; remove secrets and unrelated data.
3. Track why code exists; monitor defects and complexity; review dependencies and tests; use AI to explore options, then prioritize from actual maintenance signals.
4. Ask for assumptions and alternatives when the evidence is incomplete; do not let a confident tone substitute for a source or reproduction.
5. Review the result in the project, run proportionate checks, and keep, revise, or reject it based on observable evidence.

Worked scenario

A generated abstraction that adds indirection without reuse should be simplified when future changes and tests show it obscures behavior. A useful request separates what is known from what is suspected. For example: “Here is the sanitized behavior, relevant contract, and constraint. First identify the assumptions and propose the smallest experiment that distinguishes the likely causes. Do not change unrelated interfaces.” This asks for reasoning that can be tested rather than a patch detached from evidence.

task-brief.txt

A compact brief; replace bracketed text with project-specific evidence.

Goal: Maintain AI-assisted systems through ownership, observability, and evidence-based cleanup.
 Context: [relevant behavior, files, version, or sanitized evidence]
 Constraints: [invariants, policy, compatibility, and out-of-scope work]
 Request: Track why code exists; monitor defects and complexity; review dependencies and tests; use AI to explore options, then prioritize from actual maintenance signals.
 Check: [test, source, reproduction, or review that will verify the result]

Review and verification

- Compare the suggestion with the canonical requirement and nearby project conventions.
- Check normal, boundary, and failure behavior; inspect what the proposed change leaves untouched.
- Run the narrowest useful automated check, then broader checks when the change crosses boundaries.
- Read the complete diff and explain the result in your own words before accepting it.

A plausible answer still needs evidence

A common failure is to treat a fluent answer as proof. Maintaining and Improving AI-Assisted Codebases requires context and judgment: verify claims against code, tests, authoritative documentation, or a reproducible observation. If the tool lacks necessary context, ask a question or gather evidence instead of filling the gap with a guess.

Use AI to make the next engineering decision easier to inspect; keep the decision and its verification yours.

Working principle

Practice

Practice: Maintaining and Improving AI-Assisted Codebases Optional

Choose a maintenance signal and define a small improvement, evidence of benefit, and regression check.

1. Choose a small, permitted task related to Maintaining and Improving AI-Assisted Codebases.
2. Write the goal, relevant context, constraints, and observable acceptance criteria before consulting an AI tool.
3. Request a bounded explanation, plan, or draft; note assumptions that need confirmation.
4. Verify the result with a source, focused test, reproduction, or review checklist and record what you changed or rejected.

Constraints: Use synthetic or approved information only; never include credentials, private user data, or material you are not allowed to share.

Expected outcome: A short task brief, a reviewable AI contribution, and evidence that addresses: Choose a maintenance signal and define a small improvement, evidence of benefit, and regression check.

Knowledge check

Select your answers before checking them in the course reader. The answer key is intentionally not included in this ebook.

Question 1 (required). A developer is working on Maintaining and Improving AI-Assisted Codebases and receives a confident AI recommendation. What is the strongest next step?

- A. Apply the workflow: Track why code exists; monitor defects and complexity; review dependencies and tests; use AI to explore options, then prioritize from actual maintenance signals. Then verify the outcome against project evidence.
- B. Accept the recommendation because it is detailed and sounds technically plausible.
- C. Ask the model to repeat the same answer more confidently, then proceed without a project check.

Question 2 (required). A teammate asks you to act on this task: Maintain AI-assisted systems through ownership, observability, and evidence-based cleanup. Some requirements or evidence are missing. What is the strongest response?

- A. Identify the missing assumption, gather the smallest useful evidence, narrow the request, and verify affected behavior.
- B. Let the AI infer the missing requirements and make a broad change so work continues quickly.
- C. Ignore the uncertainty because the requested change appears small.

Designing Your Personal AI Development Workflow

Learning objectives

- Explain how Designing Your Personal AI Development Workflow supports a software-development task.
- Apply a bounded workflow that uses relevant context and makes assumptions visible.
- Evaluate and verify AI-supported work before relying on it.

Start with the engineering decision

Design a repeatable personal workflow around task risk, context, and feedback. The useful outcome is a decision or change that can be checked in the context where the software actually runs. AI can help you explore options and reduce drafting effort; it cannot supply missing requirements or take responsibility for the result.

This matters in building your ai-powered developer workflow because plausible output can still conflict with project behavior, user expectations, or operational constraints. Begin with the observed problem and define what success looks like before requesting a solution. Keep assumptions visible so a teammate can challenge them.

A practical workflow

1. State the task in terms of the behavior or decision you need: design a repeatable personal workflow around task risk, context, and feedback.
2. Supply only the relevant code, contract, evidence, and constraints; remove secrets and unrelated data.
3. Classify task; select a suitable approved tool; prepare minimal context; request bounded help; verify; record useful prompt patterns and outcomes.
4. Ask for assumptions and alternatives when the evidence is incomplete; do not let a confident tone substitute for a source or reproduction.
5. Review the result in the project, run proportionate checks, and keep, revise, or reject it based on observable evidence.

Worked scenario

A local naming question needs less context and review than an authorization change spanning several services. A useful request separates what is known from what is suspected. For example: “Here is the sanitized behavior, relevant contract, and constraint. First identify the assumptions and propose the smallest experiment that distinguishes the likely causes. Do not change unrelated interfaces.” This asks for reasoning that can be tested rather than a patch detached from evidence.

task-brief.txt

A compact brief; replace bracketed text with project-specific evidence.

Goal: Design a repeatable personal workflow around task risk, context, and feedback.

Context: [relevant behavior, files, version, or sanitized evidence]

Constraints: [invariants, policy, compatibility, and out-of-scope work]

Request: Classify task; select a suitable approved tool; prepare minimal context; request bounded help; verify; record useful prompt patterns and outcomes.

Check: [test, source, reproduction, or review that will verify the result]

Review and verification

- Compare the suggestion with the canonical requirement and nearby project conventions.
- Check normal, boundary, and failure behavior; inspect what the proposed change leaves untouched.
- Run the narrowest useful automated check, then broader checks when the change crosses boundaries.
- Read the complete diff and explain the result in your own words before accepting it.

A plausible answer still needs evidence

A common failure is to treat a fluent answer as proof. Designing Your Personal AI Development Workflow requires context and judgment: verify claims against code, tests, authoritative documentation, or a reproducible observation. If the tool lacks necessary context, ask a question or gather evidence instead of filling the gap with a guess.

Use AI to make the next engineering decision easier to inspect; keep the decision and its verification yours.

Working principle

Practice

Practice: Designing Your Personal AI Development Workflow Optional

Write a personal workflow with decision points for tool choice, context, verification depth, and stopping.

1. Choose a small, permitted task related to Designing Your Personal AI Development Workflow.
2. Write the goal, relevant context, constraints, and observable acceptance criteria before consulting an AI tool.
3. Request a bounded explanation, plan, or draft; note assumptions that need confirmation.
4. Verify the result with a source, focused test, reproduction, or review checklist and record what you changed or rejected.

Constraints: Use synthetic or approved information only; never include credentials, private user data, or material you are not allowed to share.

Expected outcome: A short task brief, a reviewable AI contribution, and evidence that addresses: Write a personal workflow with decision points for tool choice, context, verification depth, and stopping.

Knowledge check

Select your answers before checking them in the course reader. The answer key is intentionally not included in this ebook.

Question 1 (required). A developer is working on Designing Your Personal AI Development Workflow and receives a confident AI recommendation. What is the strongest next step?

- A. Apply the workflow: Classify task; select a suitable approved tool; prepare minimal context; request bounded help; verify; record useful prompt patterns and outcomes. Then verify the outcome against project evidence.
- B. Accept the recommendation because it is detailed and sounds technically plausible.
- C. Ask the model to repeat the same answer more confidently, then proceed without a project check.

Question 2 (required). A teammate asks you to act on this task: Design a repeatable personal workflow around task risk, context, and feedback. Some requirements or evidence are missing. What is the strongest response?

- A. Identify the missing assumption, gather the smallest useful evidence, narrow the request, and verify affected behavior.
- B. Let the AI infer the missing requirements and make a broad change so work continues quickly.
- C. Ignore the uncertainty because the requested change appears small.

The AI-Powered Developer: Putting Everything Together

Learning objectives

- Explain how The AI-Powered Developer: Putting Everything Together supports a software-development task.
- Apply a bounded workflow that uses relevant context and makes assumptions visible.
- Evaluate and verify AI-supported work before relying on it.

Start with the engineering decision

Synthesize the course into an end-to-end practice that keeps engineering ownership with the developer. The useful outcome is a decision or change that can be checked in the context where the software actually runs. AI can help you explore options and reduce drafting effort; it cannot supply missing requirements or take responsibility for the result.

This matters in building your ai-powered developer workflow because plausible output can still conflict with project behavior, user expectations, or operational constraints. Begin with the observed problem and define what success looks like before requesting a solution. Keep assumptions visible so a teammate can challenge them.

A practical workflow

1. State the task in terms of the behavior or decision you need: synthesize the course into an end-to-end practice that keeps engineering ownership with the developer.
2. Supply only the relevant code, contract, evidence, and constraints; remove secrets and unrelated data.
3. Understand the request; plan; choose tool; supply safe context; implement incrementally; test and review; communicate limits and learn from outcomes.
4. Ask for assumptions and alternatives when the evidence is incomplete; do not let a confident tone substitute for a source or reproduction.
5. Review the result in the project, run proportionate checks, and keep, revise, or reject it based on observable evidence.

Worked scenario

For a feature, begin with acceptance criteria, use AI to explore and draft a narrow slice, verify contracts and tests, then report decisions and remaining uncertainty. A useful request separates what is known from what is suspected. For example: “Here is the sanitized behavior, relevant contract, and constraint. First identify the assumptions and propose the smallest experiment that distinguishes the likely causes. Do not change unrelated interfaces.” This asks for reasoning that can be tested rather than a patch detached from evidence.

task-brief.txt

A compact brief; replace bracketed text with project-specific evidence.

Goal: Synthesize the course into an end-to-end practice that keeps engineering ownership with the developer.

Context: [relevant behavior, files, version, or sanitized evidence]

Constraints: [invariants, policy, compatibility, and out-of-scope work]

Request: Understand the request; plan; choose tool; supply safe context; implement incrementally; test and review; communicate limits and learn from outcomes.

Check: [test, source, reproduction, or review that will verify the result]

Review and verification

- Compare the suggestion with the canonical requirement and nearby project conventions.
- Check normal, boundary, and failure behavior; inspect what the proposed change leaves untouched.
- Run the narrowest useful automated check, then broader checks when the change crosses boundaries.
- Read the complete diff and explain the result in your own words before accepting it.

A plausible answer still needs evidence

A common failure is to treat a fluent answer as proof. The AI-Powered Developer: Putting Everything Together requires context and judgment: verify claims against code, tests, authoritative documentation, or a reproducible observation. If the tool lacks necessary context, ask a question or gather evidence instead of filling the gap with a guess.

Use AI to make the next engineering decision easier to inspect; keep the decision and its verification yours.

Working principle

Practice

Practice: The AI-Powered Developer: Putting Everything Together Optional

Apply the full loop to a realistic task and produce a brief showing decisions, AI contribution, evidence, and follow-up.

1. Choose a small, permitted task related to The AI-Powered Developer: Putting Everything Together.
2. Write the goal, relevant context, constraints, and observable acceptance criteria before consulting an AI tool.
3. Request a bounded explanation, plan, or draft; note assumptions that need confirmation.
4. Verify the result with a source, focused test, reproduction, or review checklist and record what you changed or rejected.

Constraints: Use synthetic or approved information only; never include credentials, private user data, or material you are not allowed to share.

Expected outcome: A short task brief, a reviewable AI contribution, and evidence that addresses: Apply the full loop to a realistic task and produce a brief showing decisions, AI contribution, evidence, and follow-up.

Knowledge check

Select your answers before checking them in the course reader. The answer key is intentionally not included in this ebook.

Question 1 (required). A developer is working on The AI-Powered Developer: Putting Everything Together and receives a confident AI recommendation. What is the strongest next step?

- A. Apply the workflow: Understand the request; plan; choose tool; supply safe context; implement incrementally; test and review; communicate limits and learn from outcomes. Then verify the outcome against project evidence.
- B. Accept the recommendation because it is detailed and sounds technically plausible.
- C. Ask the model to repeat the same answer more confidently, then proceed without a project check.

Question 2 (required). A teammate asks you to act on this task: Synthesize the course into an end-to-end practice that keeps engineering ownership with the developer. Some requirements or evidence are missing. What is the strongest response?

- A. Identify the missing assumption, gather the smallest useful evidence, narrow the request, and verify affected behavior.
- B. Let the AI infer the missing requirements and make a broad change so work continues quickly.
- C. Ignore the uncertainty because the requested change appears small.